



Sequential-based подходы

Занятие 7

Sequential рекомендации

История взаимодействий пользователя — это не просто набор, а строго упорядоченная последовательность.

Пользователь U : $\{i_1, i_2, i_3, \dots, i_t\}$

Задача — предсказать следующий, наиболее вероятный айтем.

Модель: $\{i_1, i_2, i_3, \dots, i_t\} \longrightarrow i_{t+1}$

Как было раньше:

User_id	Item_id	Date
5	4	2024-02-24
6	1	2024-02-24
5	3	2024-02-22
1	2	2024-02-13
5	1	2024-02-10
4	2	2024-02-08
2	3	2024-02-08
2	1	2024-02-07
....		
1	4	2023-12-31



U/I	item 1	item2	item 3	item 4
User 1	0	1	0	1
User 2	1	0	1	0
User 3	0	1	0	1
User 4	0	1	0	0
User 5	1	0	1	1
User 6	1	1	0	1
User 7	0	1	1	0
User 8	1	0	0	1

Как будет теперь:

User_id	Item_id	Date
5	4	2024-02-24
6	1	2024-02-24
5	3	2024-02-22
1	2	2024-02-13
5	1	2024-02-10
4	2	2024-02-08
2	3	2024-02-08
2	1	2024-02-07
....		
1	4	2023-12-31



User_1: [4,2]

User_2: [1,3]

User_4: [2]

User_5: [1,3,4]

User_6: [1]



Время

Sequential рекомендации

Зачем это нужно?

1. Учет временной динамики

Sequential рекомендации

Зачем это нужно?

1. **Учет временной динамики**
2. **Причинно-следственные паттерны**

Sequential рекомендации

Зачем это нужно?

1. **Учет временной динамики**
2. **Причинно-следственные паттерны**
3. **Повторные взаимодействия**

Sequential рекомендации

Зачем это нужно?

1. **Учет временной динамики**
2. **Причинно-следственные паттерны**
3. **Повторные взаимодействия**
4. **Недавние события часто важнее старых**

Sequential рекомендации

Параллель из NLP (задачи next token prediction, language modeling):

токены $\rightarrow$ item IDs

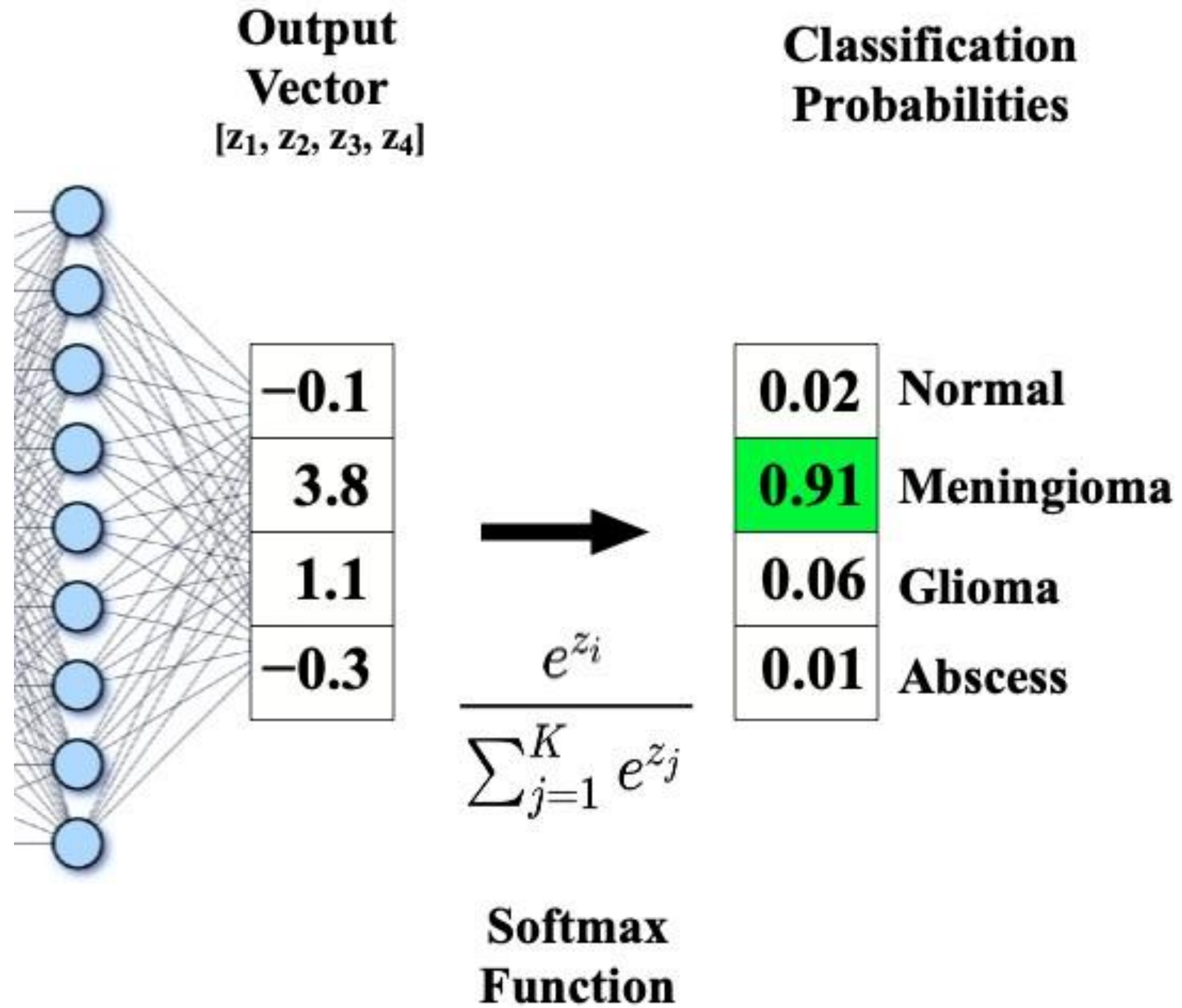
предложение $\rightarrow$ пользовательская сессия (история взаимодействий)

next token prediction $\rightarrow$ next item prediction

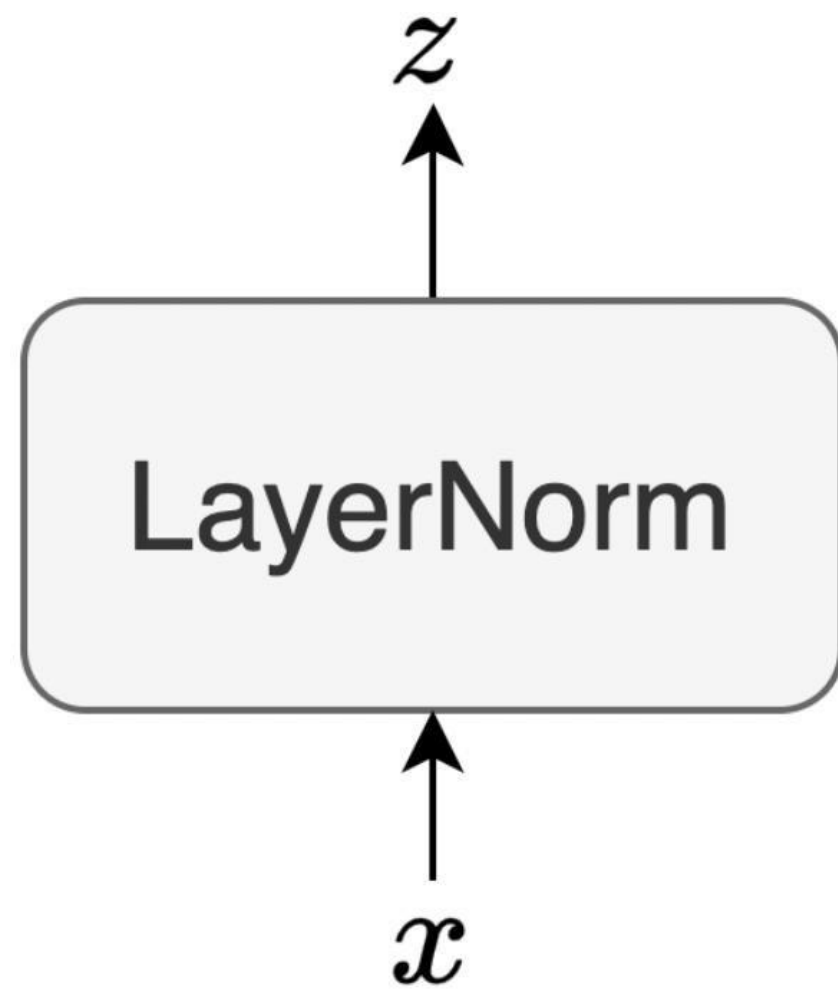
$$P(i_{t+1} \mid i_1, i_2, \dots, i_t)$$

Вспоминаем основы из DL

Softmax



LayerNorm



$$z_i = \alpha \cdot \frac{x_i - \hat{\mu}}{\hat{\sigma}} + \beta, \quad i \in \{1, 2, \dots, d\},$$

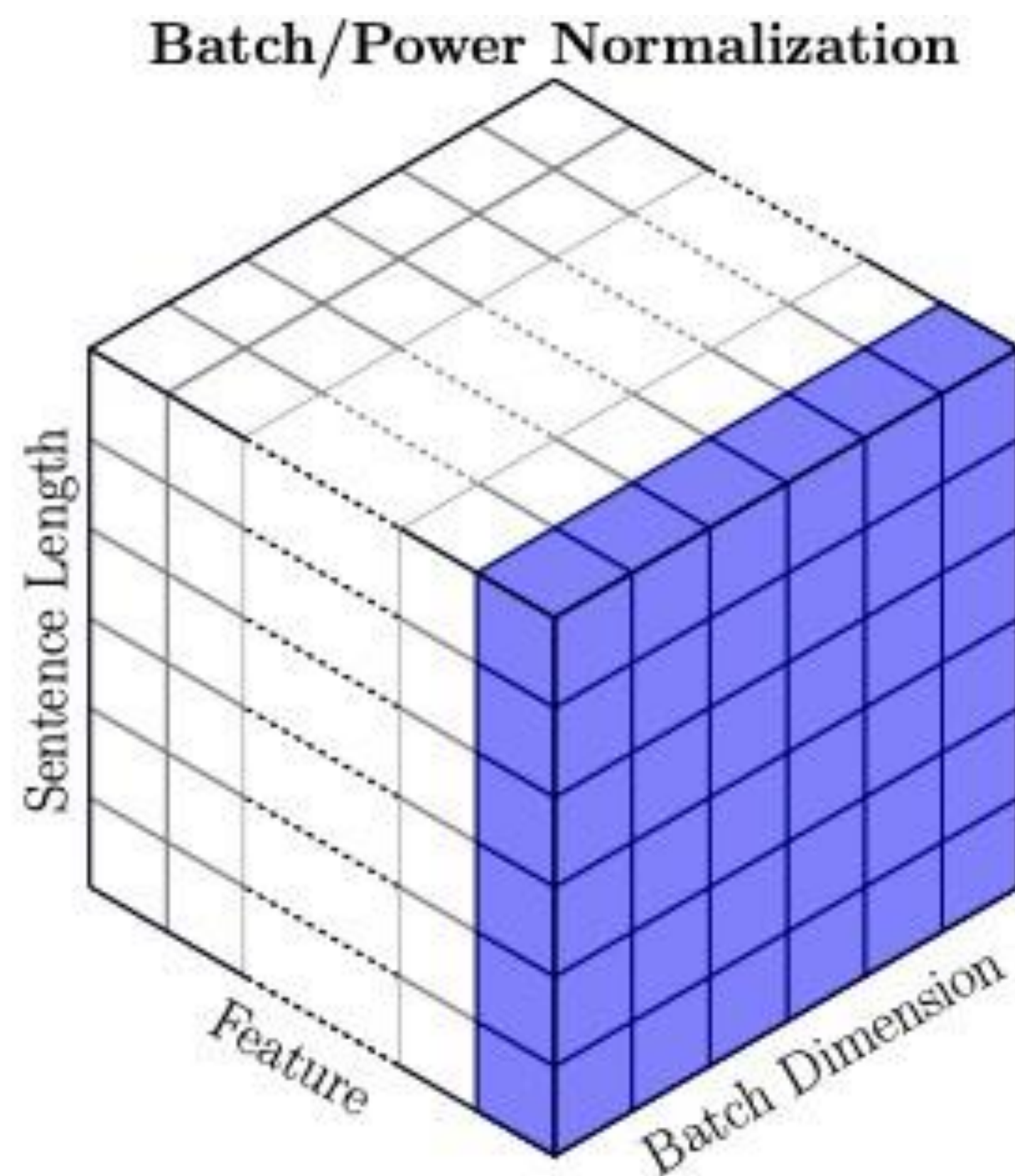
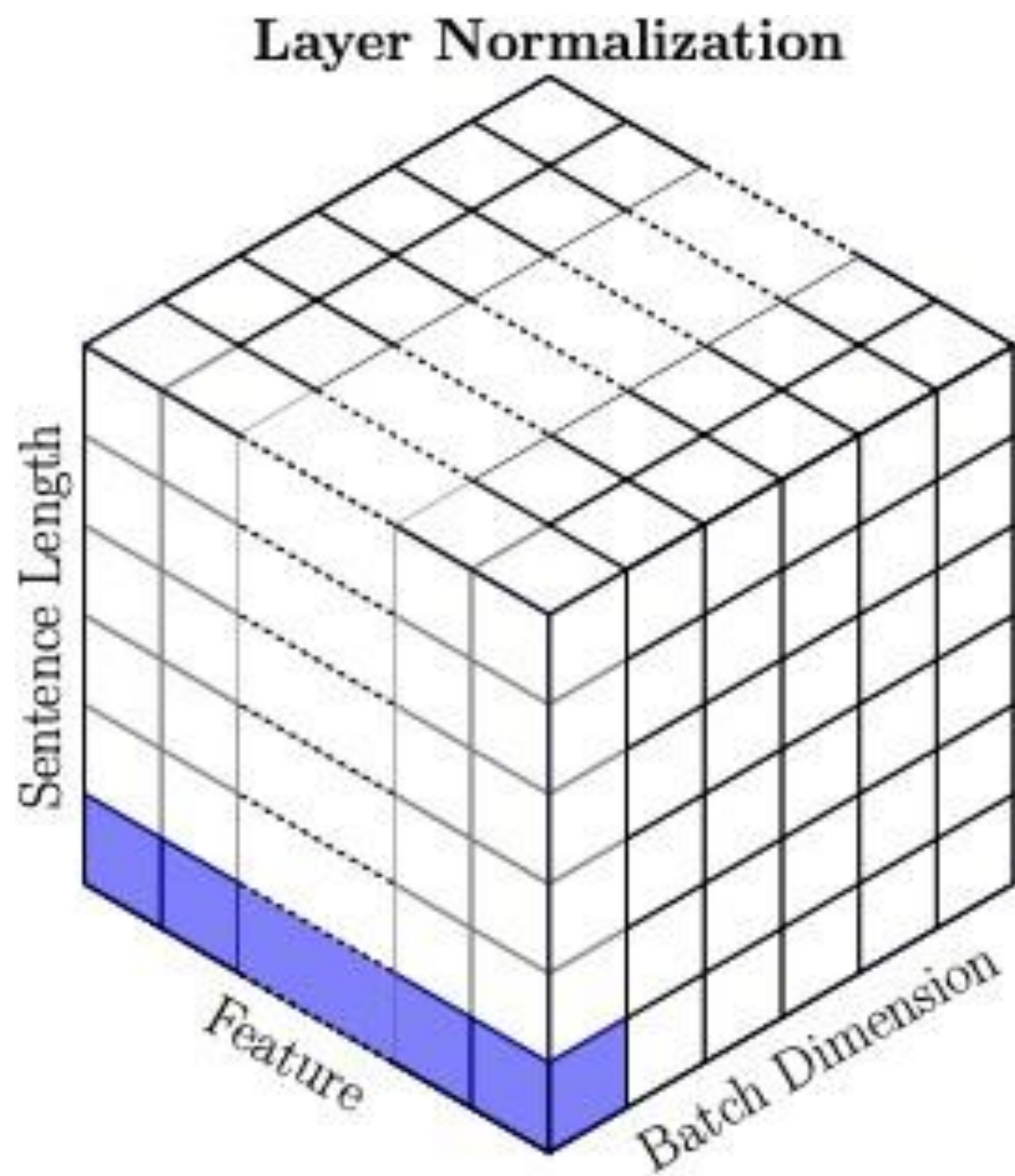
$$\hat{\mu} = \frac{\sum_{i=1}^d x_i}{d},$$

$$\hat{\sigma} = \sqrt{\frac{\sum_{i=1}^d (x_i - \hat{\mu})^2}{d}}$$

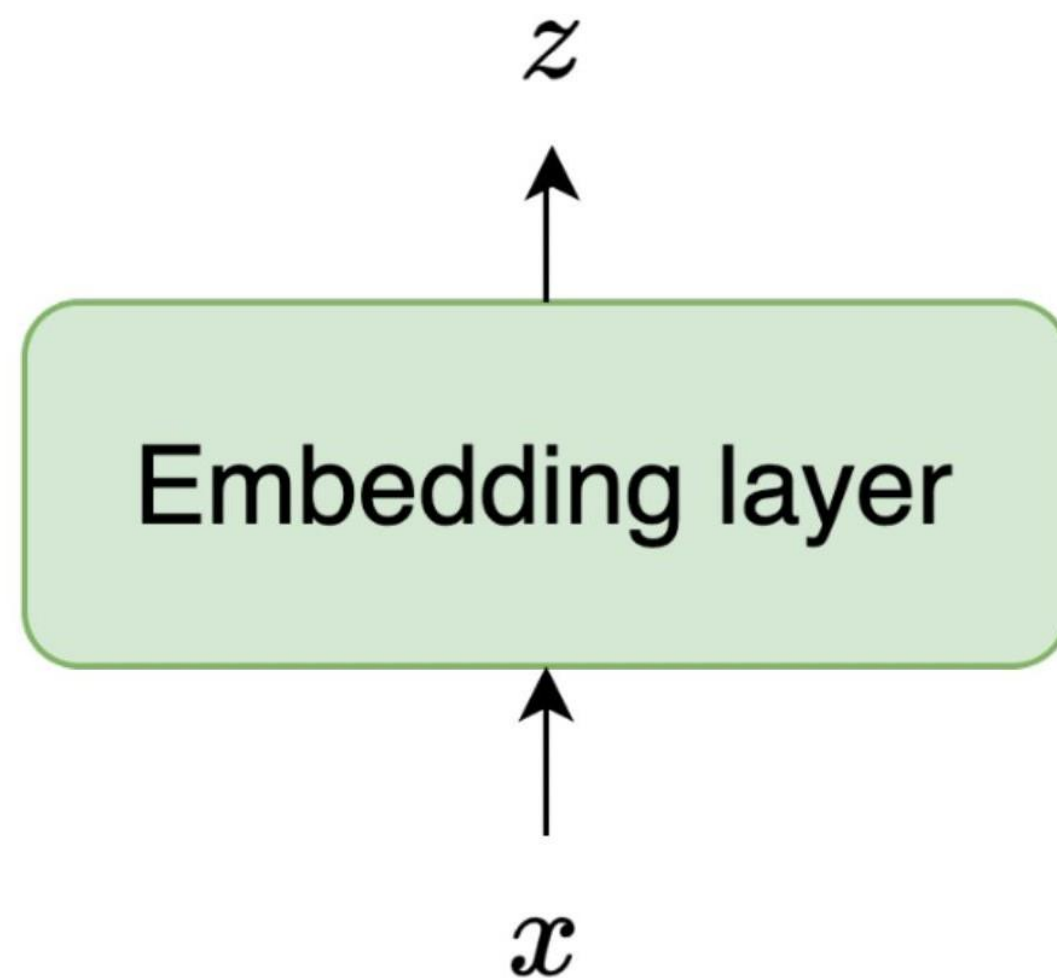
$$x, z \in \mathbb{R}^{1 \times d},$$

$$\alpha, \beta \in \mathbb{R}$$

BatchNorm vs LayerNorm



Эмбединги



$$z = (W_{x,1}, W_{x,2}, \dots, W_{x,d}),$$

$$x \in \{0, 1, 2, \dots, V - 1\},$$

$$z \in \mathbb{R}^{1 \times d},$$

$$W \in \mathbb{R}^{V \times d}$$

Пусть x может принимать одно из V значений (от 0 до $V - 1$).

Слой эмбедингов превращает x в вектор.

То есть мы каждому айтому сопоставляем некоторый вектор из d -мерного пространства.

Эмбеддинги

Айтем	Индекс
бананы	0
молоко	1
хлеб	2
колбаса	3

0.32	-0.21	0.22	-0.23	0.01	-0.12
0.13	0.172	0.12	0.45	-0.42	0.65
0.01	0.89	0.65	-0.75	-0.31	-0.64
-0.98	1.23	-0.93	-0.85	0.31	0.33

Какой эмбеддинг у хлеба?

Эмбеддинги

Айтем	Индекс
бананы	0
молоко	1
хлеб	2
колбаса	3

0.32	-0.21	0.22	-0.23	0.01	-0.12
0.13	0.172	0.12	0.45	-0.42	0.65
0.01	0.89	0.65	-0.75	-0.31	-0.64
-0.98	1.23	-0.93	-0.85	0.31	0.33

Какой эмбеддинг у хлеба?

Эмбединги

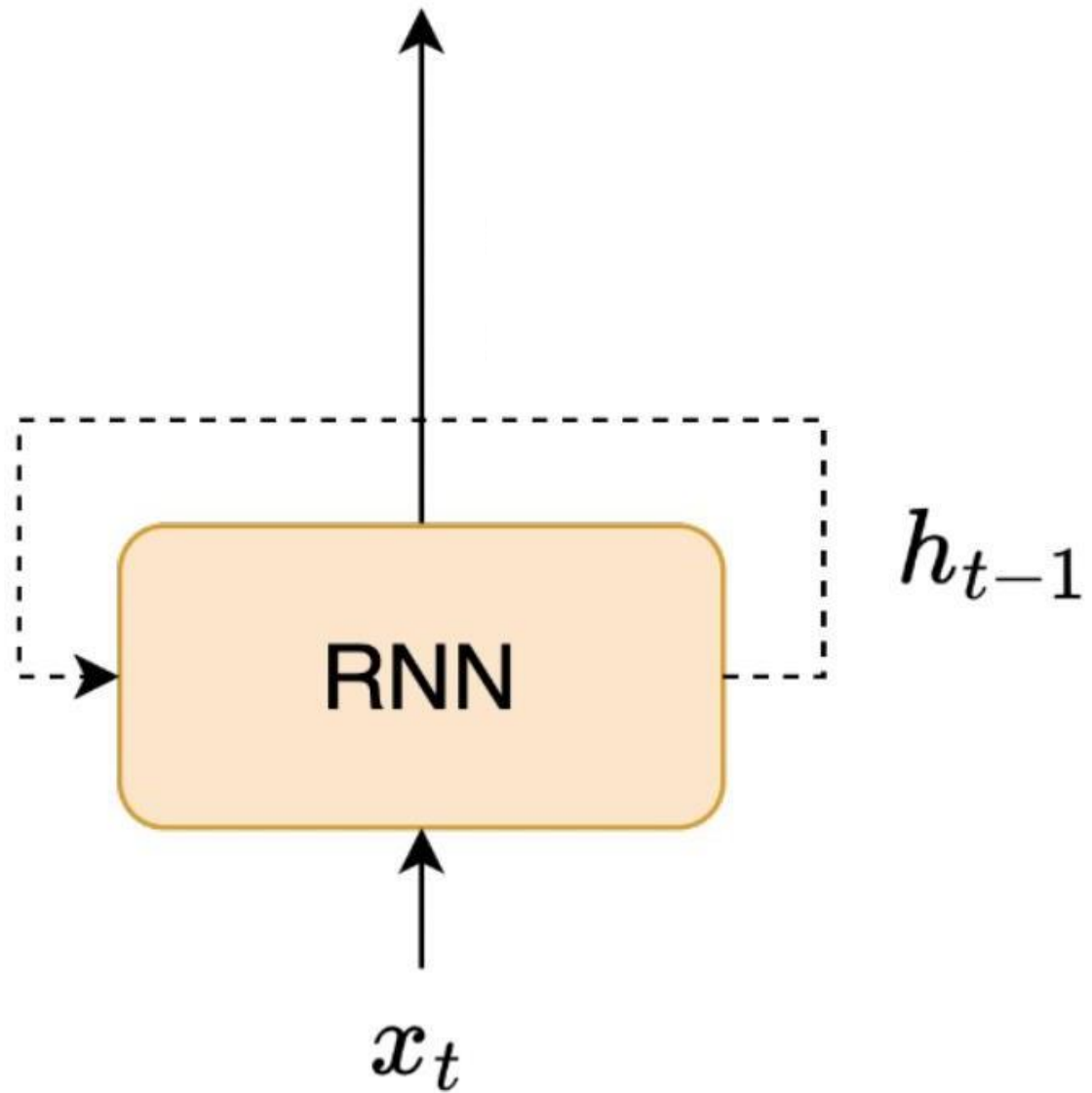
Айтем	Индекс
хлеб	0
колбаса	1
бананы	2
молоко	3

0.32	-0.21	0.22	-0.23	0.01	-0.12
0.13	0.172	0.12	0.45	-0.42	0.65
0.01	0.89	0.65	-0.75	-0.31	-0.64
-0.98	1.23	-0.93	-0.85	0.31	0.33

Порядок **не важен**, главное чтобы он **не менялся**

Recurrent Neural Network (RNN)

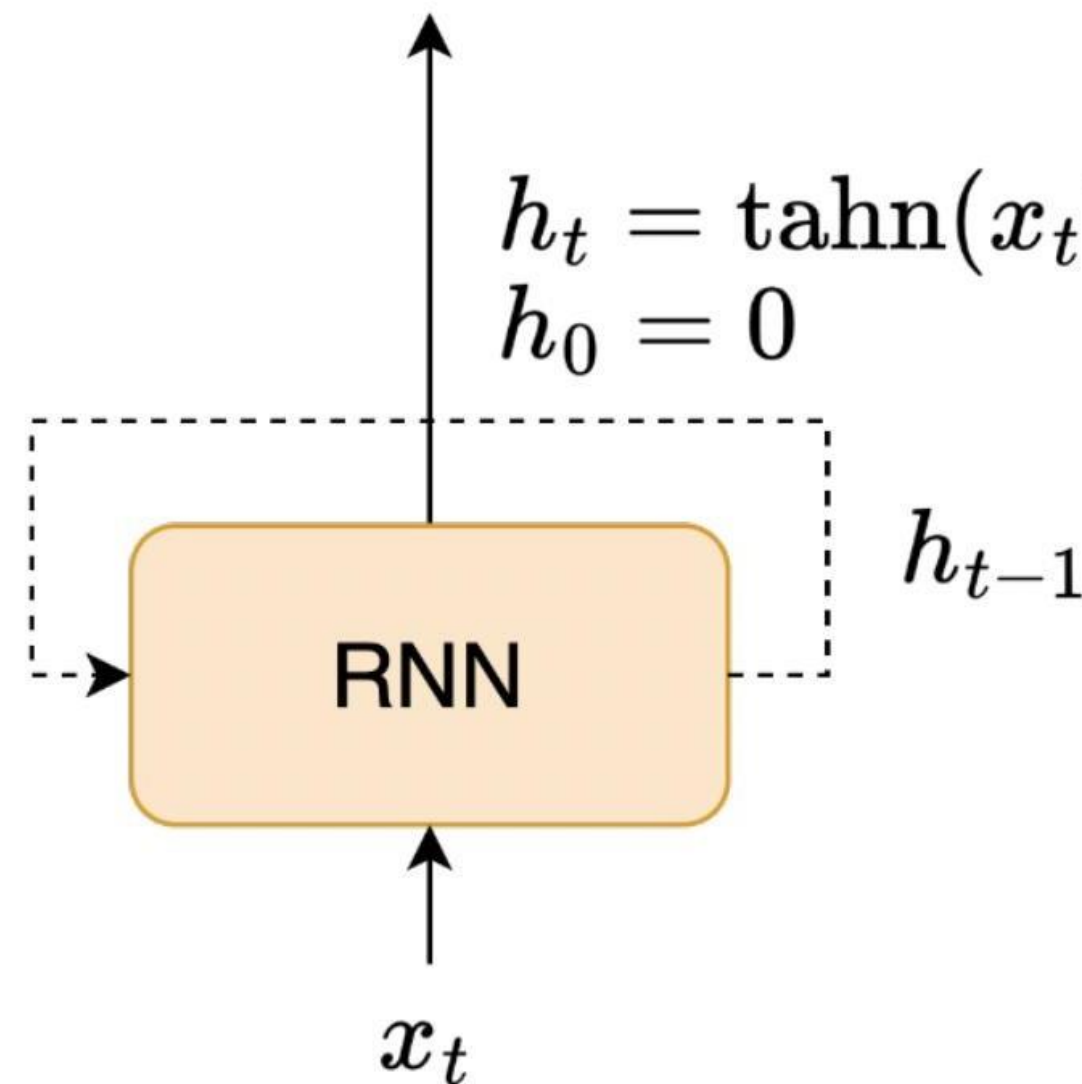
Цель: обработать последовательность $x_1, x_2, \dots, x_L$ зависимых наблюдений **нефиксированной** длины.



Recurrent Neural Network (RNN)

Цель: обработать последовательность $x_1, x_2, \dots, x_L$ зависимых наблюдений **нефиксированной** длины.

$$\begin{aligned}x_t &\in \mathbb{R}^{1 \times d_x}, \\h_t, b &\in \mathbb{R}^{1 \times d_h}, \\W_x &\in \mathbb{R}^{d_x \times d_h}, \\W_h &\in \mathbb{R}^{d_h \times d_h}\end{aligned}$$

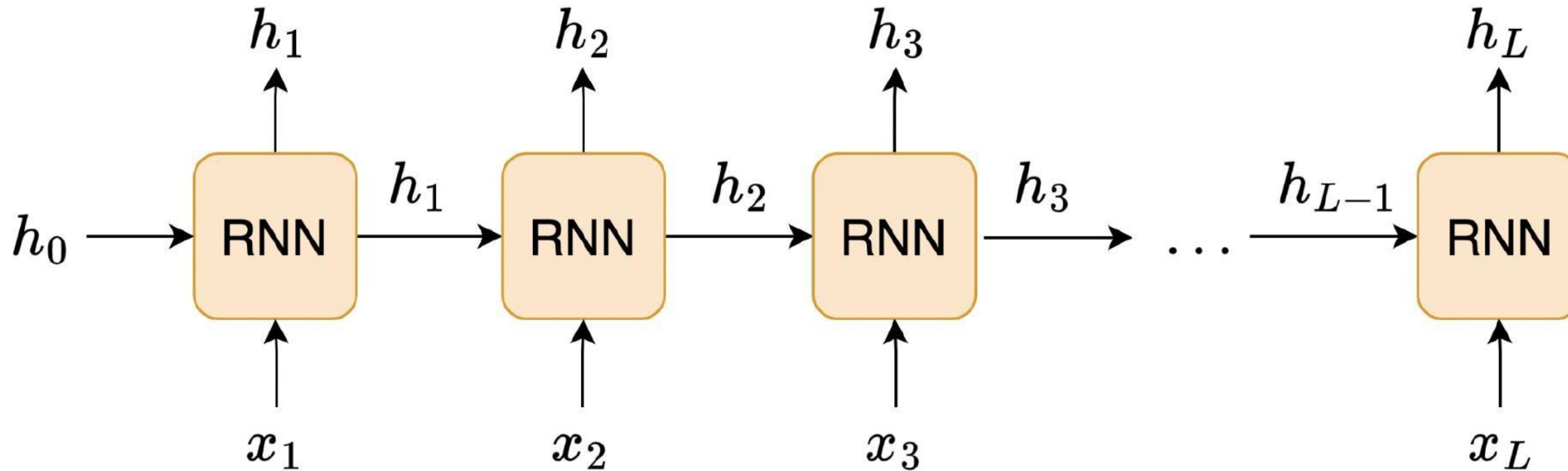


$$\begin{aligned}h_t &= \text{tanh}(x_t W_x + h_{t-1} W_h + b), \\h_0 &= 0\end{aligned}$$

} Скрытые состояния

- Состояние h_t можно рассматривать как внутреннюю память модели на шаге t .
- Состояние h_t неявно зависит от всех $x_1, \dots, x_t$, то есть хранит информацию о входах до шага t включительно.

Развернутая RNN



$$x_t \in \mathbb{R}^{1 \times d_x},$$

$$h_t, b \in \mathbb{R}^{1 \times d_h},$$

$$W_x \in \mathbb{R}^{d_x \times d_h},$$

$$W_h \in \mathbb{R}^{d_h \times d_h}$$

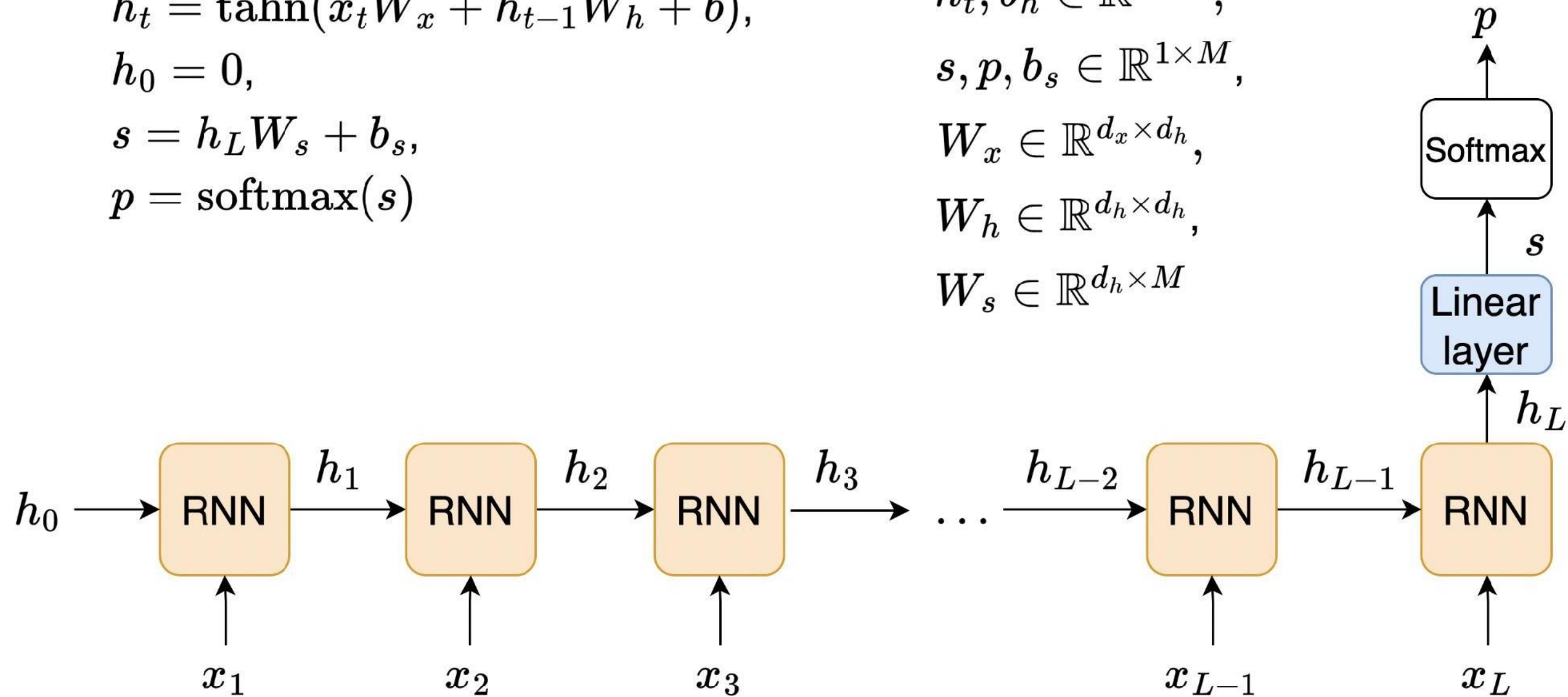
$$h_t = \text{tanh}(x_t W_x + h_{t-1} W_h + b),$$

$$h_0 = 0$$

RNN для рекомендаций

$$\begin{aligned}h_t &= \tanh(x_t W_x + h_{t-1} W_h + b), \\h_0 &= 0, \\s &= h_L W_s + b_s, \\p &= \text{softmax}(s)\end{aligned}$$

$$\begin{aligned}x_t &\in \mathbb{R}^{1 \times d_x}, \\h_t, b_h &\in \mathbb{R}^{1 \times d_h}, \\s, p, b_s &\in \mathbb{R}^{1 \times M}, \\W_x &\in \mathbb{R}^{d_x \times d_h}, \\W_h &\in \mathbb{R}^{d_h \times d_h}, \\W_s &\in \mathbb{R}^{d_h \times M}\end{aligned}$$



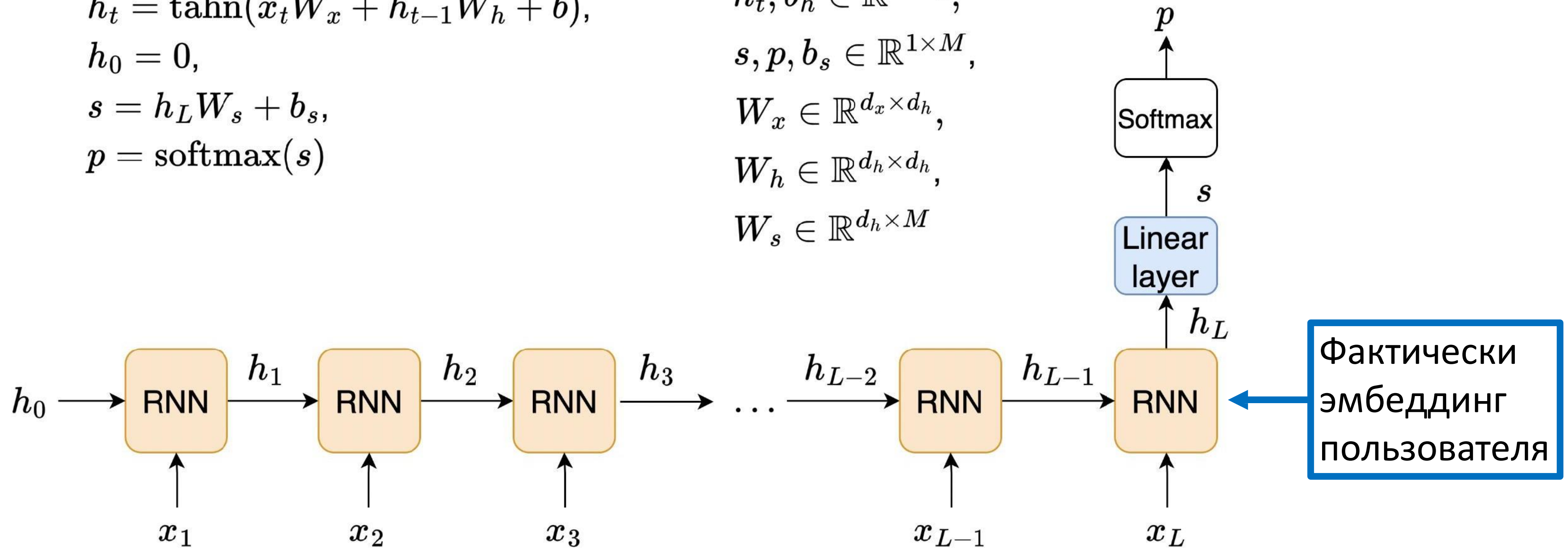
s - ЛОГИТЫ,

p - предсказанное распределение (M классов)

RNN для рекомендаций

$$\begin{aligned}h_t &= \tanh(x_t W_x + h_{t-1} W_h + b), \\h_0 &= 0, \\s &= h_L W_s + b_s, \\p &= \text{softmax}(s)\end{aligned}$$

$$\begin{aligned}x_t &\in \mathbb{R}^{1 \times d_x}, \\h_t, b_h &\in \mathbb{R}^{1 \times d_h}, \\s, p, b_s &\in \mathbb{R}^{1 \times M}, \\W_x &\in \mathbb{R}^{d_x \times d_h}, \\W_h &\in \mathbb{R}^{d_h \times d_h}, \\W_s &\in \mathbb{R}^{d_h \times M}\end{aligned}$$



s - ЛОГИТЫ,

p - предсказанное распределение (M классов)

Серебряная пуля? Нет

- Не бесконечное capacity

Серебряная пуля? Нет

- Не бесконечное capacity
- Плохо параллелится

Серебряная пуля? Нет

- Не бесконечное capacity
- Плохо параллелится
- Затухают градиенты

Серебряная пуля? Нет

- Не бесконечное capacity
- Плохо параллелится
- Затухают градиенты
- Взрываются градиенты

Серебряная пуля? Нет

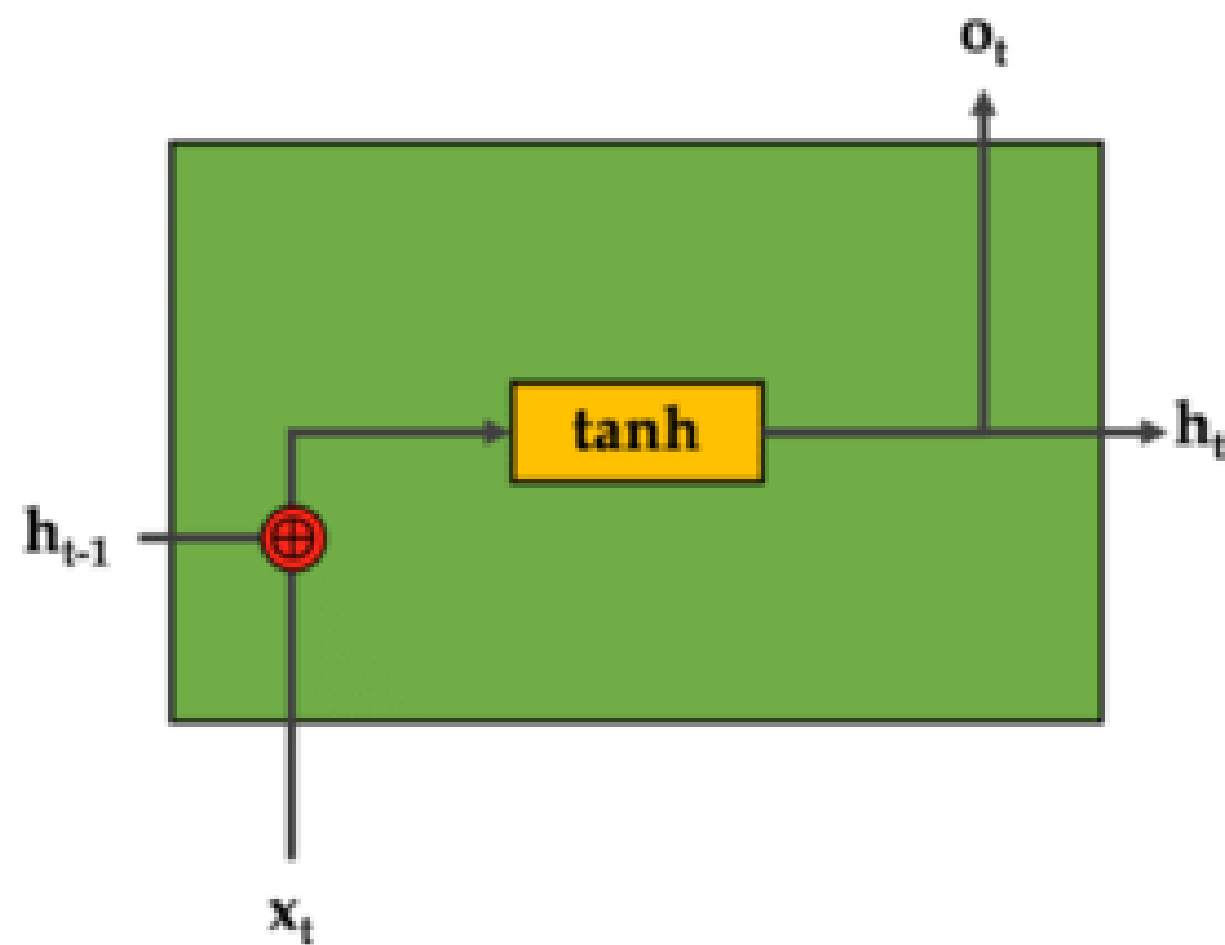
- Не бесконечное capacity
- Плохо параллелится
- Затухают градиенты
- Взрываются градиенты
- Не может запоминать длинные последовательности

А можно как-то исправить?

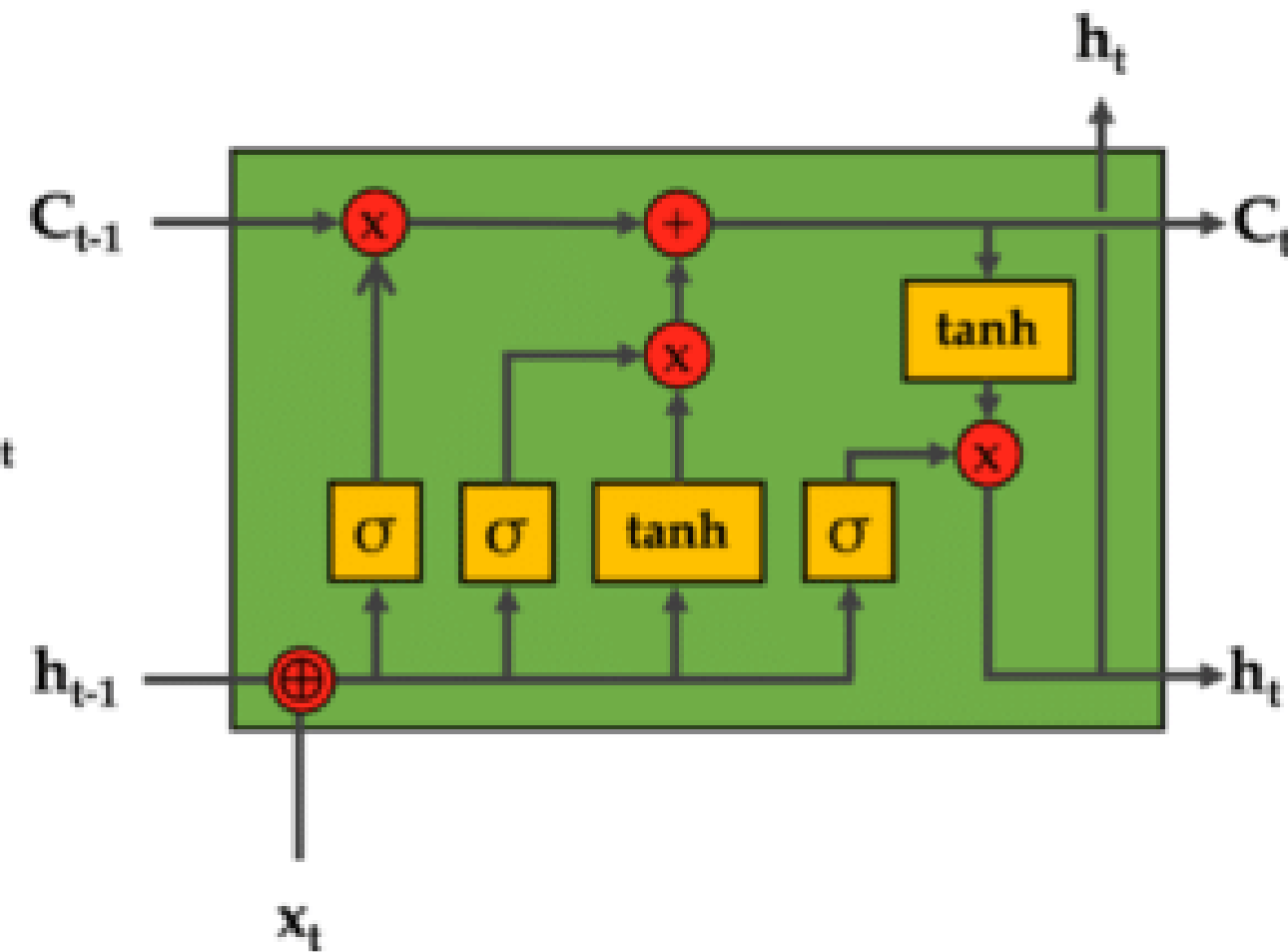
- Не бесконечное capacity
- Плохо параллелится – можно упоротся и распараллелить
- Затухают градиенты – есть модификации классической RNN
- Взрываются градиенты – можем их просто обрезать
- Не может запоминать длинные последовательности

Long-Short Term Memory & Gated Recurrent Unit

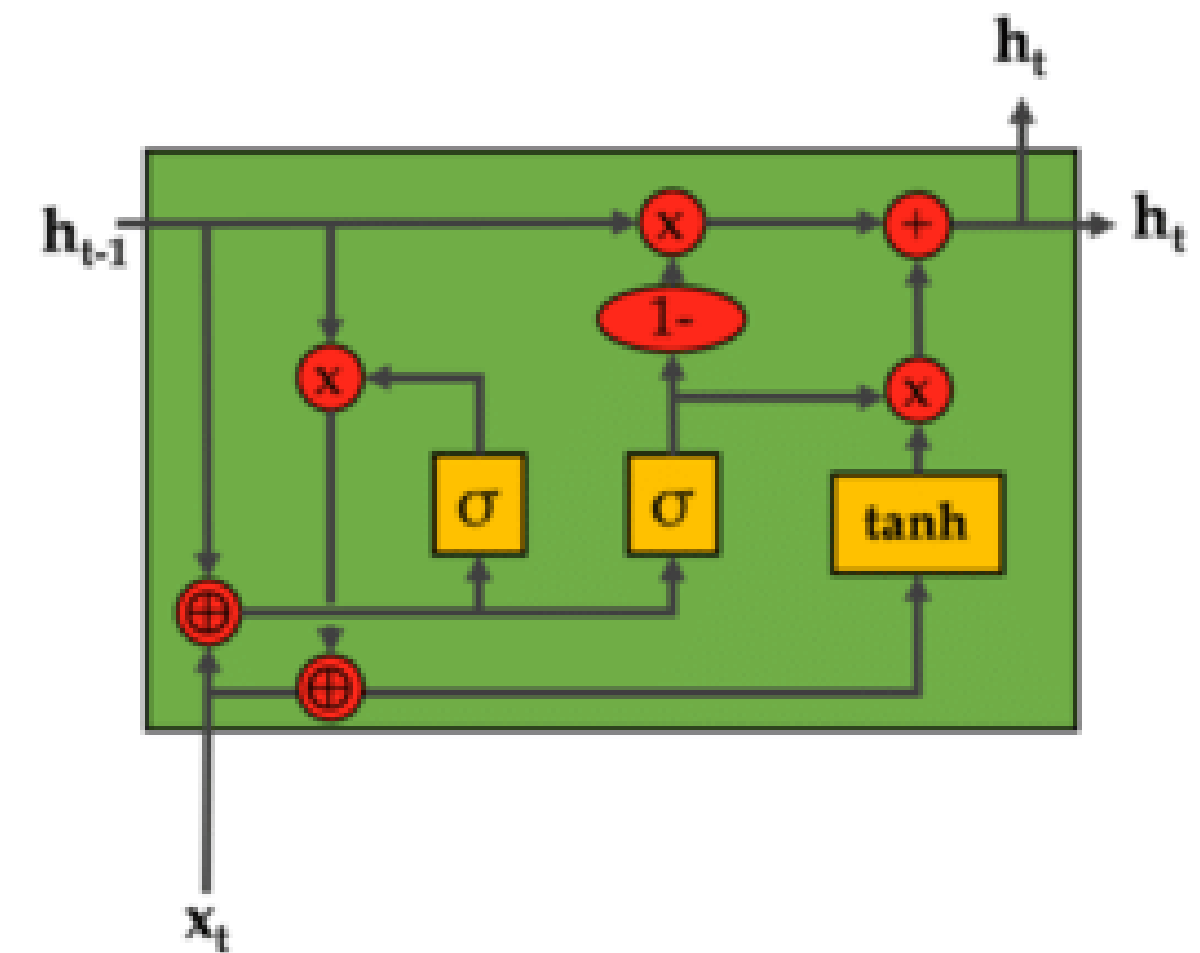
RNN



LSTM



GRU



σ sigmoid function

$\tanh$ hyperbolic tangent function

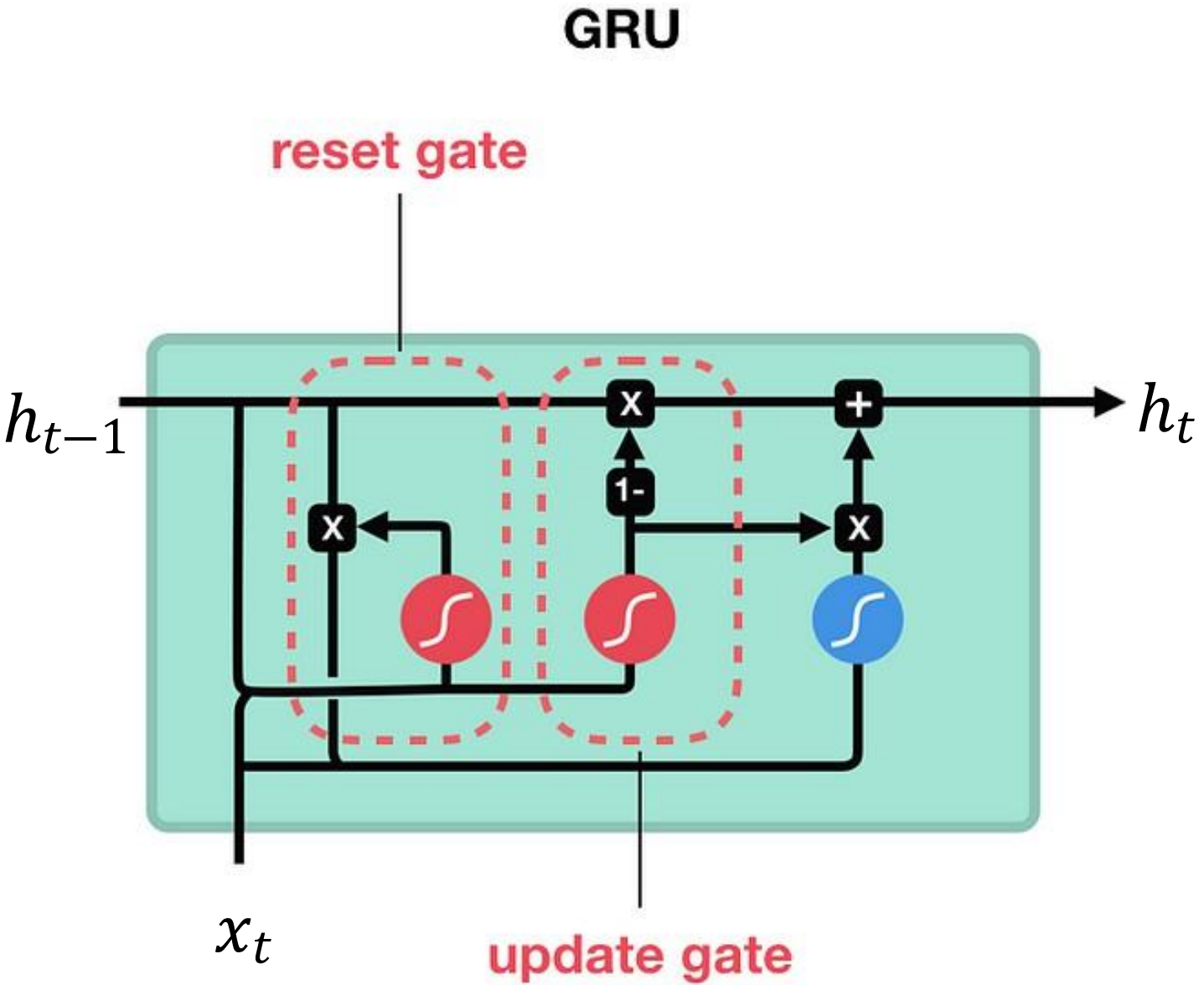
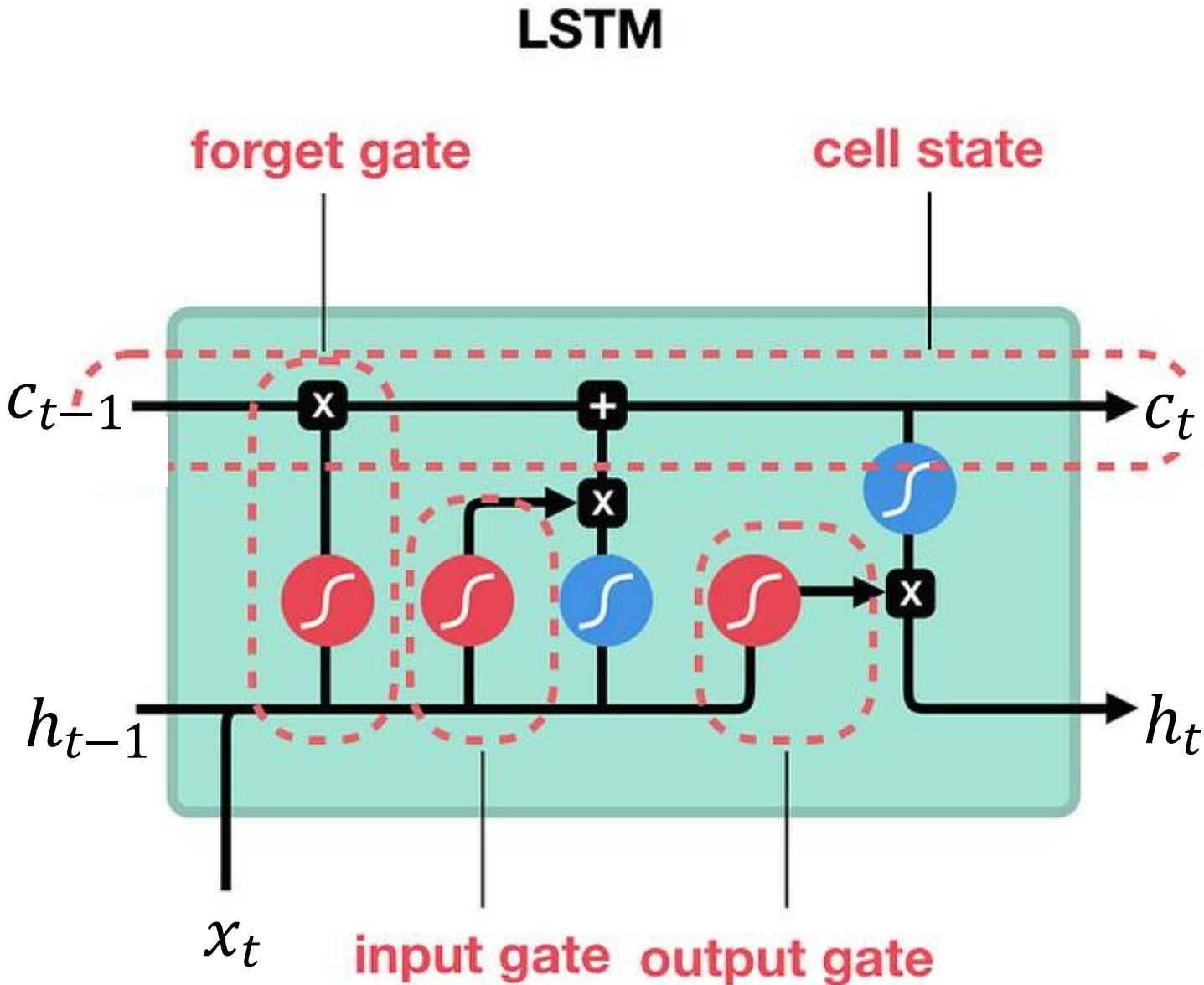
1- subtract from one

$+$ pointwise addition

$\times$ pointwise multiplication

$\oplus$ vector concatenation

LSTM & GRU



sigmoid



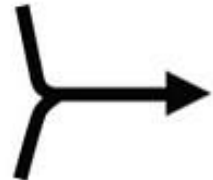
tanh



pointwise
multiplication

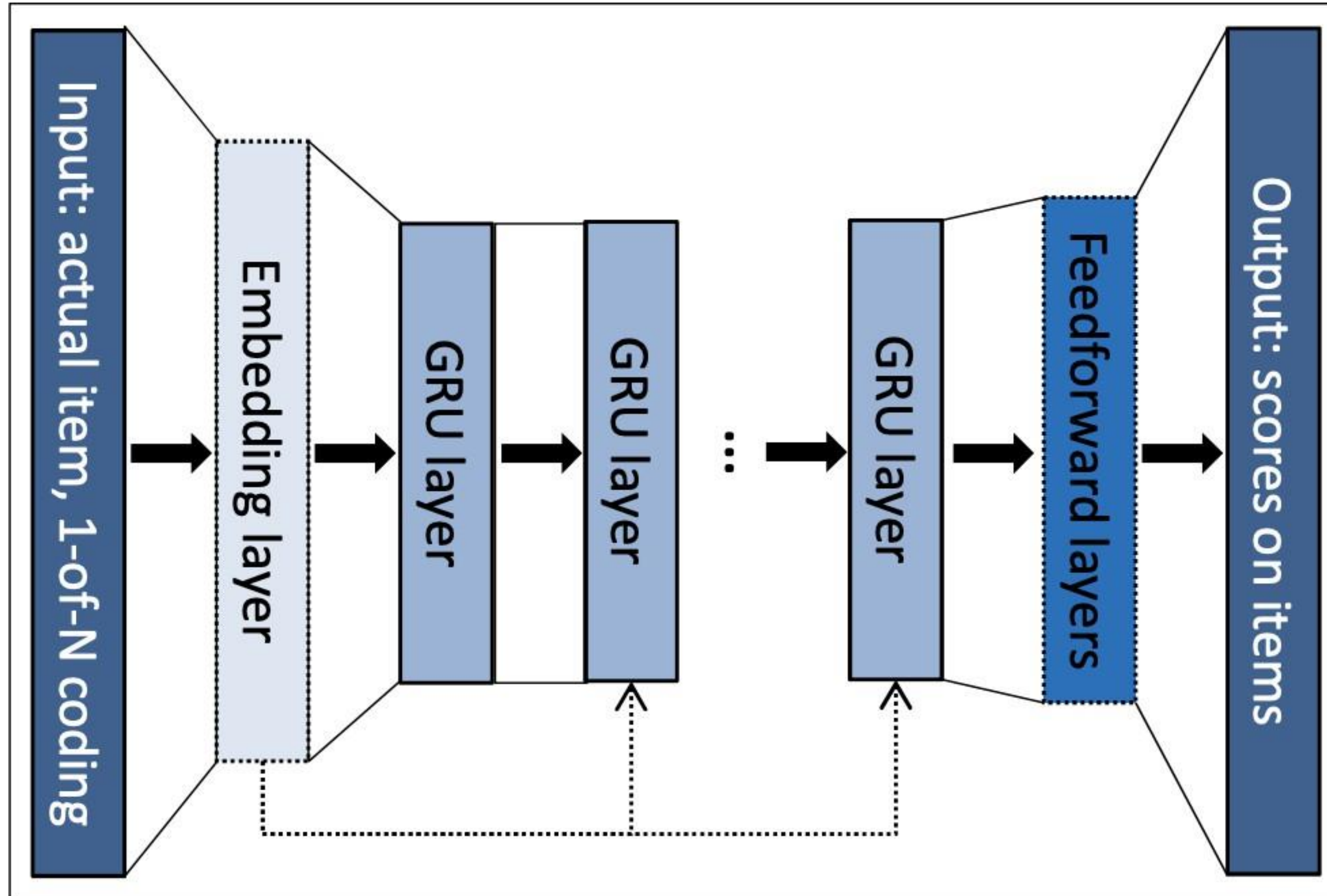


pointwise
addition

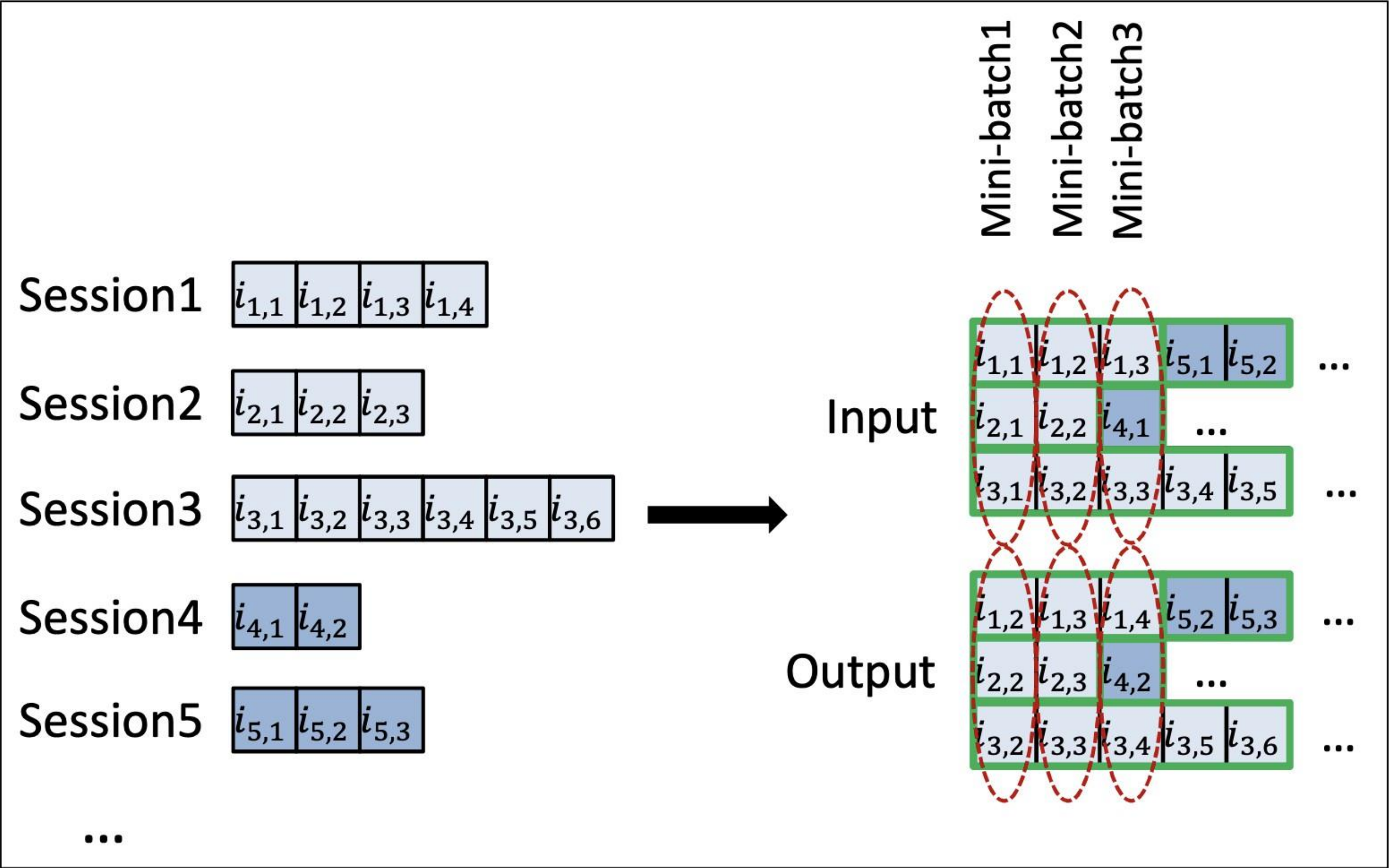


vector
concatenation

GRU4Rec



GRU4Rec



GRU4Rec

Временной шаг	Слот 1 (Сессия A)	Слот 2 (Сессия B)	Слот 3 (Сессия C)
t=1	Событие A1	Событие B1	Событие C1
t=2	Событие A2	Событие B2	Событие C2
t=3	Событие A3	Сессия B закончилась	Событие C3
t=4	Событие A4	Начинается Сессия D (D1)	Событие C4
t=5	Сессия A закончилась	Событие D2	Событие C5

Внимание!



Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

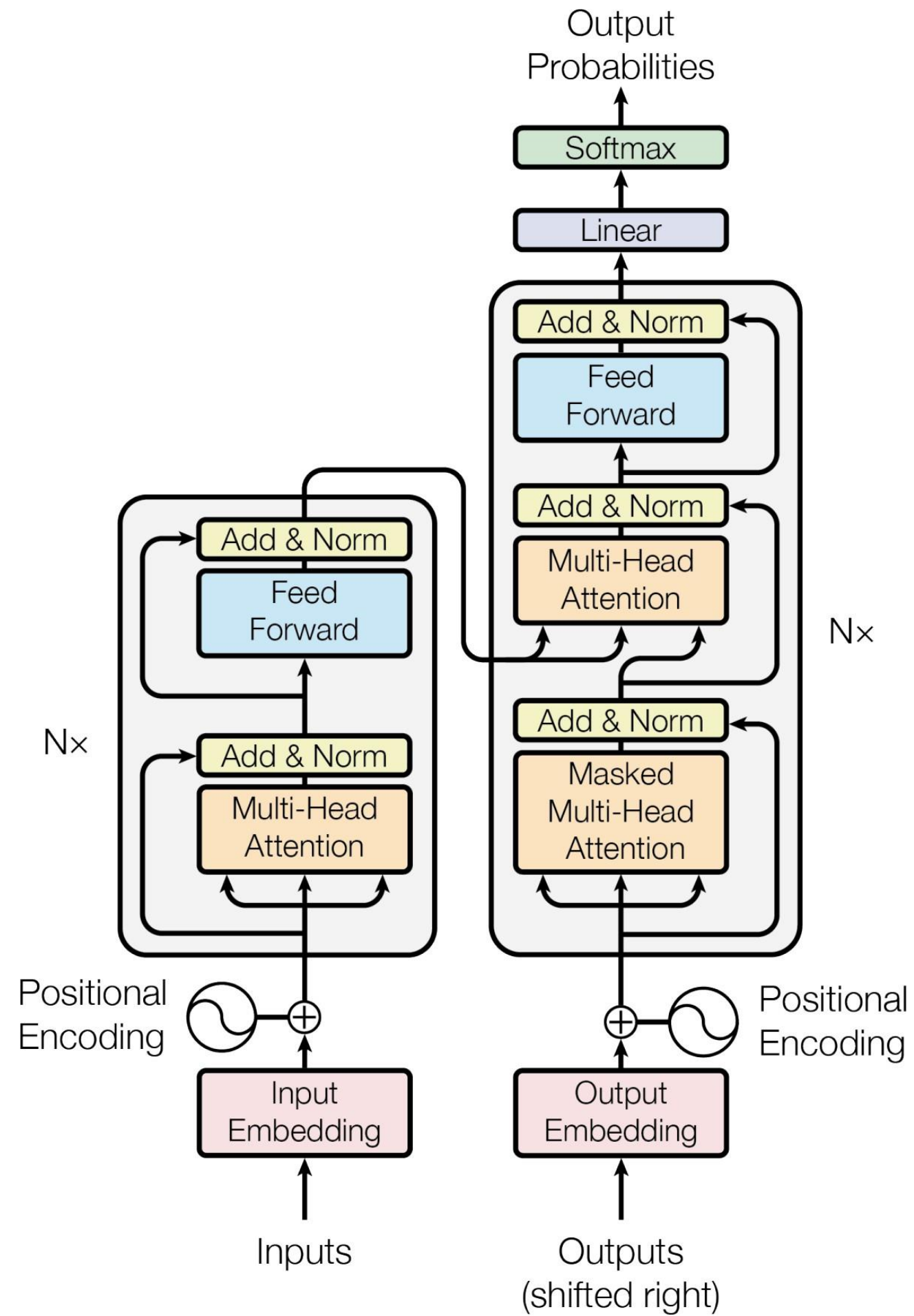
Illia Polosukhin* ‡
illia.polosukhin@gmail.com

2017

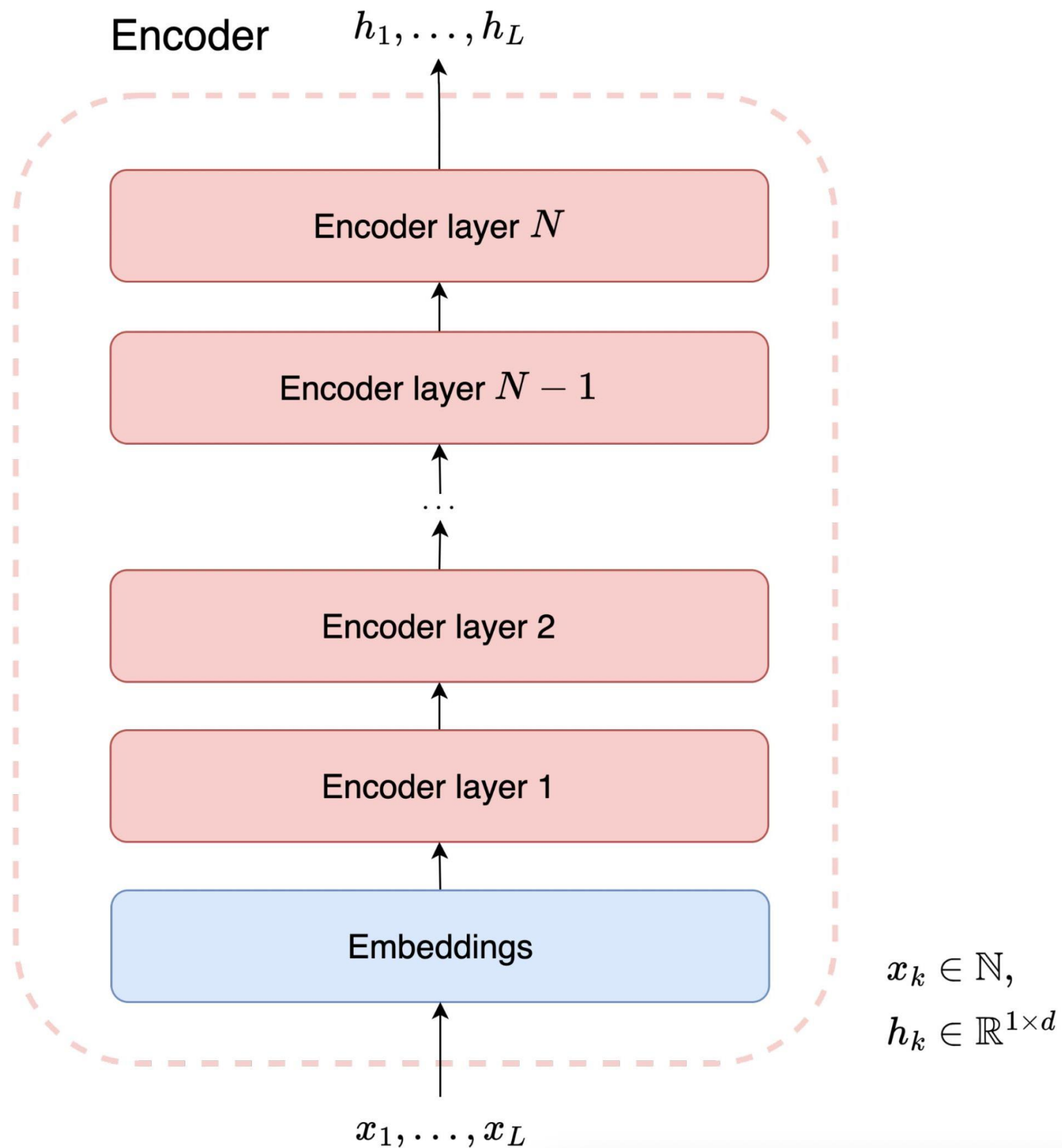
Внимание!

Архитектура, пришедшая на смену RNN:

- Хорошо параллелится
- Внутри нет рекуррентности и связанной с ней проблем, используется только механизм внимания
- Имеем доступ ко всей информации (нет узкого места, как в RNN)



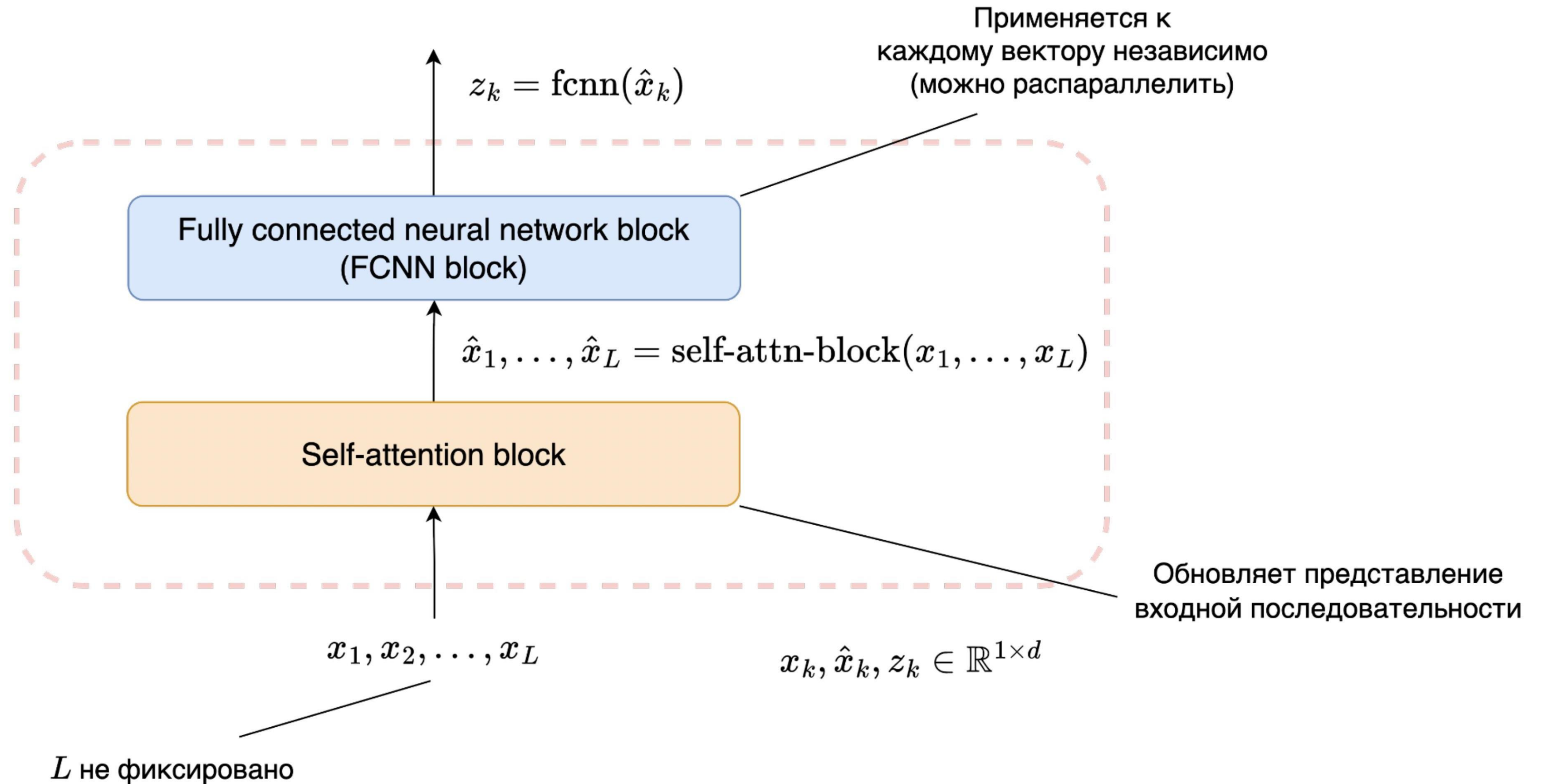
Кодировщик



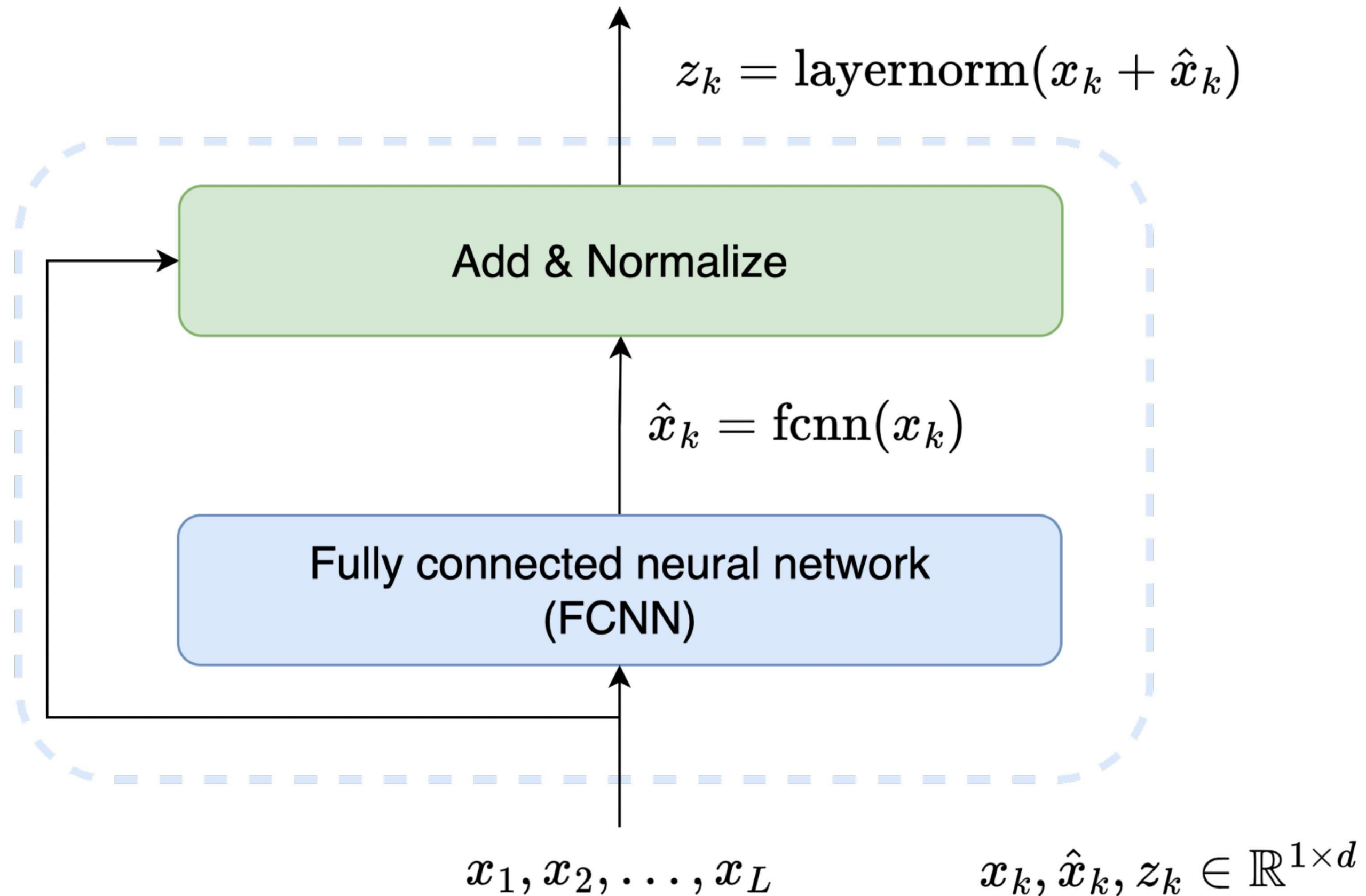
Кодировщик:

- Принимает на вход последовательность токенов
- Вычисляет эмбединги входных токенов (слой embeddings)
- Применяет последовательно N однотипных преобразований (encoder layer)

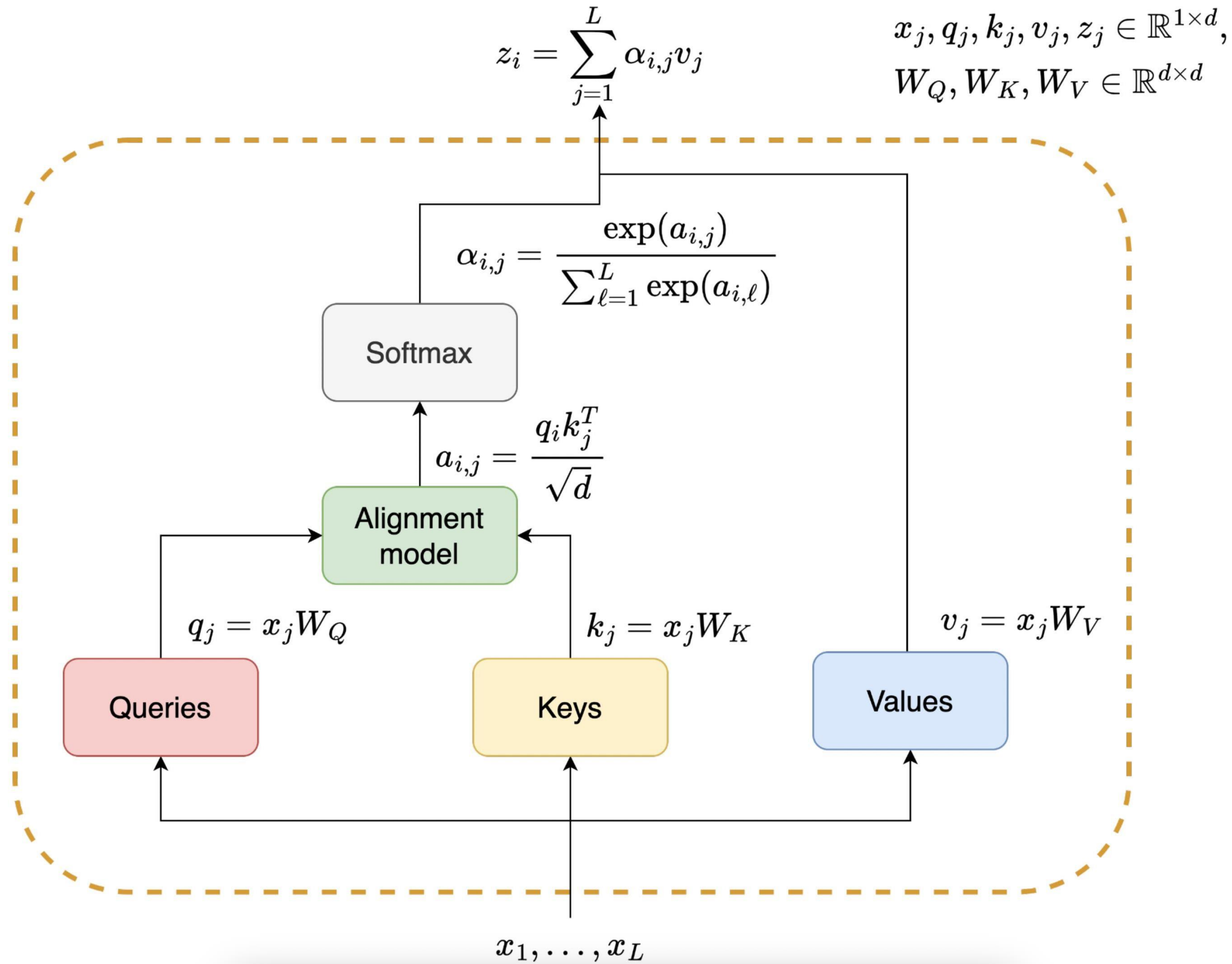
Слой кодировщика



Fully connected neural network

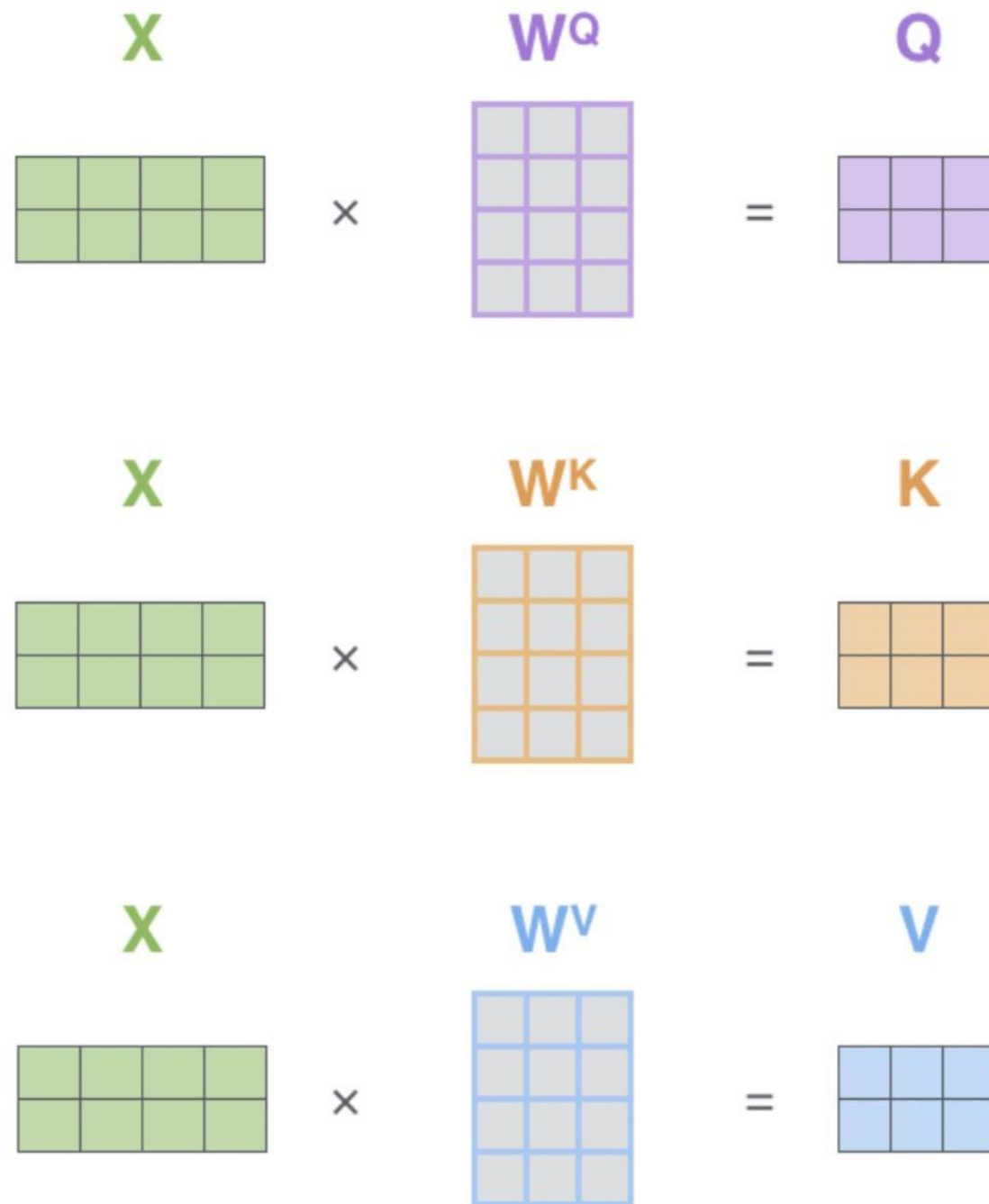


Self Attention



Self Attention

Получаем матрицы Q, K, V:



Считаем self-attention в матричной форме:

$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) V$$

$= Z$

The diagram illustrates the calculation of the self-attention matrix Z. It shows the Q matrix (purple 2x3 grid) multiplied by the K matrix (orange 2x3 grid) to produce a 2x3 grid, which is then divided by $\sqrt{d_k}$ and passed through a softmax function. The result is then multiplied by the V matrix (blue 2x3 grid) to produce the final self-attention matrix Z (pink 2x3 grid).

Query, Key, Value

Пример для AirPods

Query (запрос)	"Что я ищу?" — текущий товар ищет связи с другими	$Q = [0.7, 0.3, 0.4]$ (техника Apple, беспроводное)
Key (ключ)	"Какая у меня бирка?" — описание каждого товара	$K_1 = [0.7, 0.3, 0.2]$ (iPhone), $K_2 = [0.3, 0.6, 0.2]$ (чехол), $K_3 = [0.6, 0.2, 0.5]$ (сам AirPods)
Value (значение)	"Что я могу дать?" — полезное содержимое товара	$V_1 = [0.5, 0.4, 0.6, 0.2]$, $V_2 = [0.1, 0.8, 0.3, 0.1]$, $V_3 = [0.6, 0.3, 0.5, 0.4]$

Шаг 1. Сырые веса:

$$\text{score}_3 = [Q_3 \cdot K_1, Q_3 \cdot K_2, Q_3 \cdot K_3] = [0.66, 0.47, 0.68]$$

Расчет первой компоненты нового вектора

$$\begin{aligned} \text{score}_{31} &= Q_3 \cdot K_1 = [0.7, 0.3, 0.4] \cdot [0.7, 0.3, 0.2] \\ &= 0.7 \times 0.7 + 0.3 \times 0.3 + 0.4 \times 0.2 \\ &= 0.49 + 0.09 + 0.08 = 0.66 \end{aligned}$$

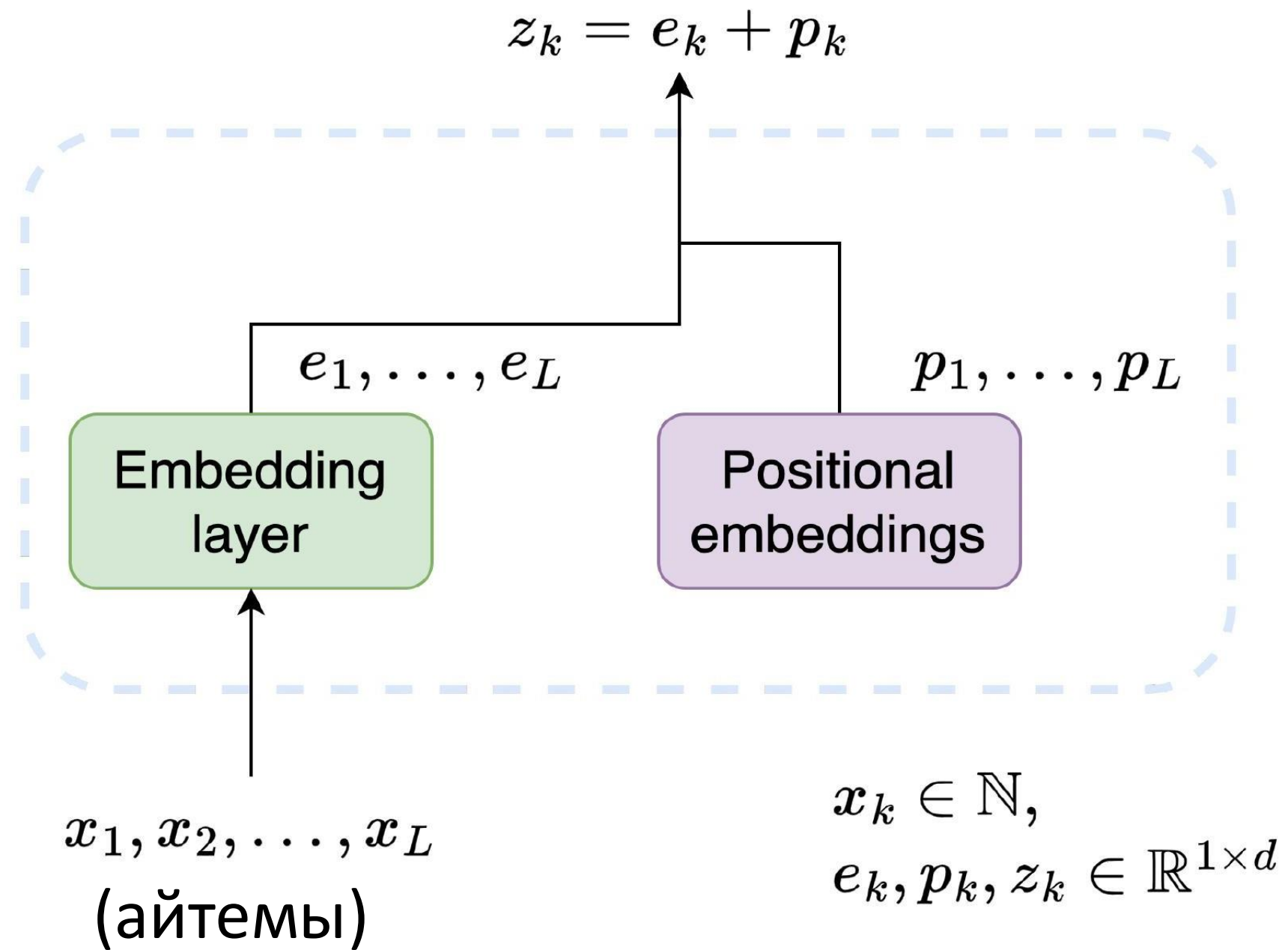
Шаг 2. Веса внимания (softmax):

$$\text{weights} = [0.35, 0.29, 0.36]$$

Шаг 3. Новый эмбединг:

$$Z_3 = 0.35 \cdot V_1 + 0.29 \cdot V_2 + 0.36 \cdot V_3 = [0.42, 0.48, 0.48, 0.24]$$

А как учесть позицию?



Преимущества Transformer для RecSys



1. Моделирование контекста через Self-Attention позволяет учесть какие из взаимодействий с айтемами важные, а какие - шум.
2. Для рекомендаций порядок важен из-за динамики интересов, причинно-следственных связей, поэтому учитываем эту информацию в positional embeddings
3. Унифицированная токенизация позволяет добавить больше известной информации

[SESSION_START, ТОВАР_123, ТИП_ПРОСМОТР, ЗАПРОС, "sci", "-fi", ТОВАР_456, ТИП_КЛИК, ...]

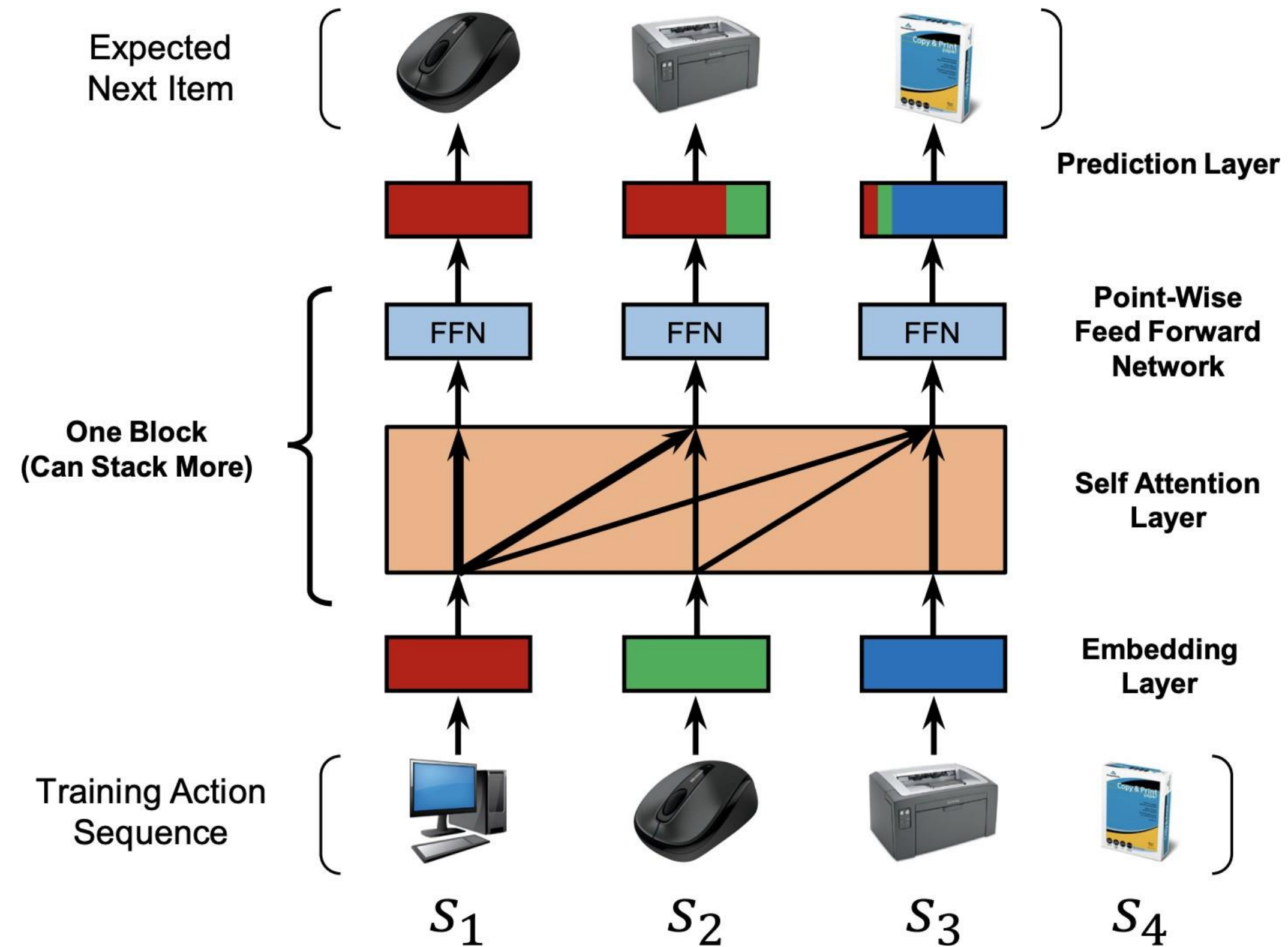
Основные sequential модели в RecSys:

SASRec & BERT4Rec

Self-Attentive Sequential Recommendation

Wang-Cheng Kang, Julian McAuley
UC San Diego
{wckang,jmcauley}@ucsd.edu

SASRec



SASRec

SASRec — это **авторегрессионная модель**

- **декодер** Трансформера с **односторонним** (causal) **вниманием**
- при предсказании элемента на позиции k, модель "видит" только предыдущие элементы 1...k-1
- Использует задачу **Sequence Shifting** (или Sequence Continuation)

Этапы:

1. Подготовка обучающих примеров

[A] → predict B
[A, B] → predict C
[A, B, C] → predict D

2. Embedding слой $x_t = E_{\text{item}} + E_{\text{pos}}$

3. Self-Attention с causal mask Self-Attention → Add & Norm → FFN → Add & Norm → **h**

4. Transformer блоки

5. Предсказание - dot product с embedding-матрицей всех айтемов

SASRec: Детали

- Shared эмбединги айтемов на **входе** и на **выходе**
- BCE Loss
- Обучаемые positional embeddings
- Маскируем аттеншн: не можем смотреть в будущее

$$L_{BCE} = - \sum_{\mathcal{S}^u \in \mathcal{S}} \sum_{t \in [1, 2, \dots, n]} \left[\log(\sigma(r_{o_t, t})) + \sum_{j \notin \mathcal{S}^u} \log(1 - \sigma(r_{j, t})) \right].$$

Note that we ignore the terms where $o_t = \text{<pad>}$.

SASRec: Детали

Как посчитать релевантность i -го айтема:

- Прогоняем всю последовательность через модель
- Берем последний hidden layer
- Умножаем на $embedding_i$

SASRec: Интересности

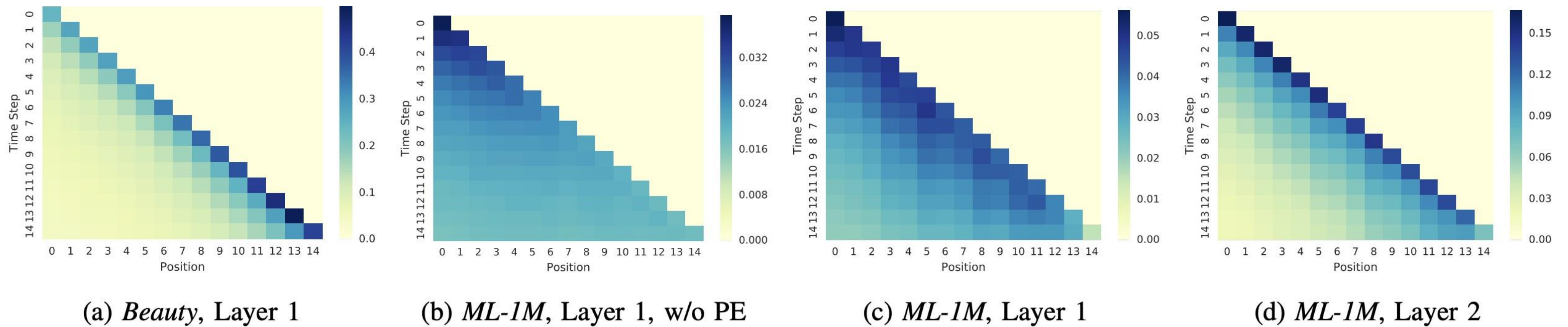


Figure 4: Visualizations of average attention weights on positions at different time steps. For comparison, the heatmap of a first-order Markov chain based model would be a diagonal matrix.

SASRec: Интересности

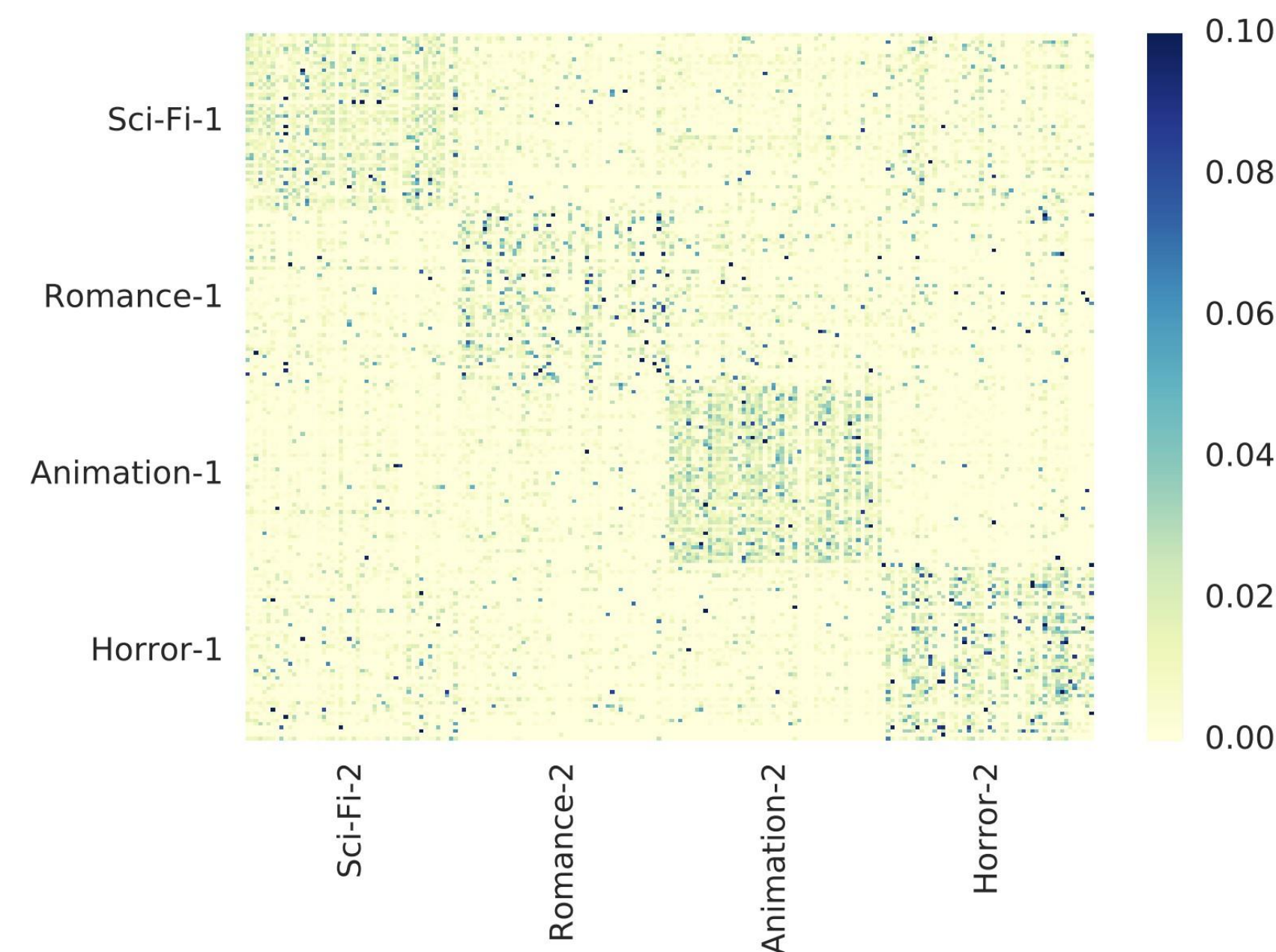


Figure 5: Visualization of average attention between movies from four categories. This shows our model can uncover items' attributes, and assigns larger weights between similar items.

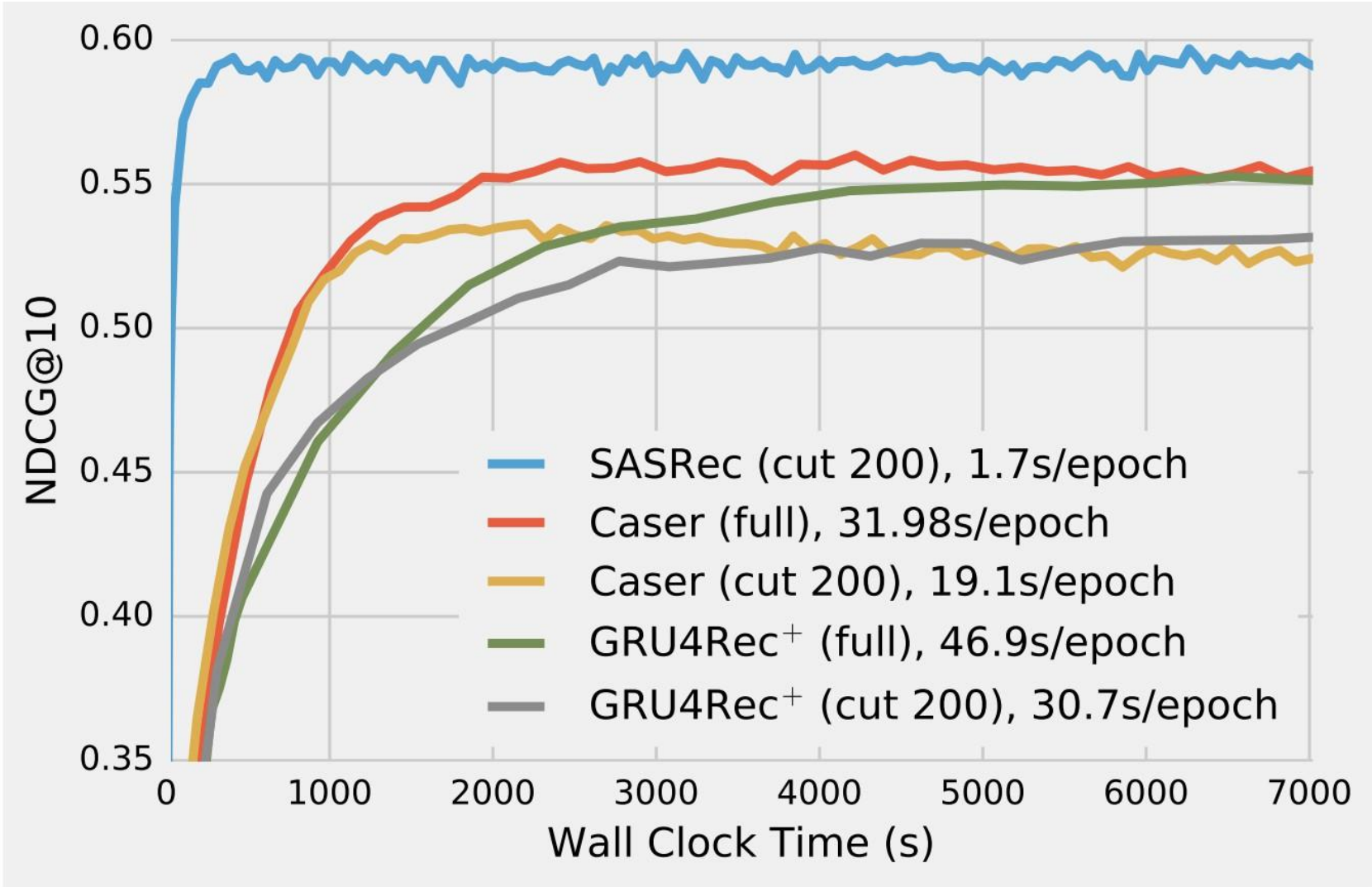


Figure 3: Training efficiency on *ML-1M*. SASRec is an order of magnitude faster than CNN/RNN-based recommendation methods in terms of training time per epoch and in total.

SASRec: Интересности

Architecture	<i>Beauty</i>	<i>Games</i>	<i>Steam</i>	<i>ML-1M</i>
(0) Default	0.3142	0.5360	0.6306	0.5905
(1) Remove PE	0.3183	0.5301	0.6036	0.5772
(2) Unshared IE	0.2437↓	0.4266↓	0.4472↓	0.4557↓
(3) Remove RC	0.2591↓	0.4303↓	0.5693	0.5535
(4) Remove Dropout	0.2436↓	0.4375↓	0.5959	0.5801
(5) 0 Block ($b=0$)	0.2620↓	0.4745↓	0.5588↓	0.4830↓
(6) 1 Block ($b=1$)	0.3066	0.5408	0.6202	0.5653
(7) 3 Blocks ($b=3$)	0.3078	0.5312	0.6275	0.5931
(8) Multi-Head	0.3080	0.5311	0.6272	0.5885

SASRec: Качество

Dataset	Metric	(a) PopRec	(b) BPR	(c) FMC	(d) FPMC	(e) TransRec	(f) GRU4Rec	(g) GRU4Rec ⁺	(h) Caser	(i) SASRec	Improvement vs. (a)-(e) (f)-(h)	
<i>Beauty</i>	Hit@10	0.4003	0.3775	0.3771	0.4310	<u>0.4607</u>	0.2125	0.3949	0.4264	0.4854	5.4%	13.8%
	NDCG@10	0.2277	0.2183	0.2477	0.2891	<u>0.3020</u>	0.1203	0.2556	0.2547	0.3219	6.6%	25.9%
<i>Games</i>	Hit@10	0.4724	0.4853	0.6358	0.6802	<u>0.6838</u>	0.2938	0.6599	0.5282	0.7410	8.5%	12.3%
	NDCG@10	0.2779	0.2875	0.4456	0.4680	<u>0.4557</u>	0.1837	<u>0.4759</u>	0.3214	0.5360	14.5%	12.6%
<i>Steam</i>	Hit@10	0.7172	0.7061	0.7731	0.7710	0.7624	0.4190	<u>0.8018</u>	0.7874	0.8729	13.2%	8.9%
	NDCG@10	0.4535	0.4436	0.5193	0.5011	0.4852	0.2691	<u>0.5595</u>	0.5381	0.6306	21.4%	12.7%
<i>ML-1M</i>	Hit@10	0.4329	0.5781	0.6986	0.7599	0.6413	0.5581	0.7501	<u>0.7886</u>	0.8245	8.5%	4.6%
	NDCG@10	0.2377	0.3287	0.4676	0.5176	0.3969	0.3381	0.5513	<u>0.5538</u>	0.5905	14.1%	6.6%

SASRec: инженерные советы

- **Padding / truncation**
- User A $\rightarrow$ [A, B]
User B $\rightarrow$ [A, B, C, D, E, F, G]
User C $\rightarrow$ [A]
- **Attention mask внутри Трансформера**
- [PAD, PAD, A, B, C]
[0, 0, 1, 1, 1]
- **Loss mask**
- **Sequence length limit**
- **Sampling strategy**
- **Batch construction**
- **Memory bottlenecks**

BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer

Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang
Alibaba Group, Beijing, China
{ofey.sf, yanhan.lj, joshuawu.wujian, changhua.pch, hc.lx, santong.oww, jiangpeng.jp}@alibaba-inc.com

ABSTRACT

Modeling users’ dynamic preferences from their historical behaviors is challenging and crucial for recommendation systems. Previous methods employ sequential neural networks to encode users’ historical interactions from left to right into hidden representations for making recommendations. Despite their effectiveness, we argue that such left-to-right unidirectional models are sub-optimal due to the limitations including: **a)** unidirectional architectures restrict the power of hidden representation in users’ behavior sequences; **b)** they often assume a rigidly ordered sequence which is not always practical. To address these limitations, we proposed a sequential recommendation model called **BERT4Rec**, which employs the deep bidirectional self-attention to model user behavior sequences. To avoid the information leakage and efficiently train the bidirectional model, we adopt the Cloze objective to sequential recommendation, predicting the random masked items in the sequence by jointly conditioning on their left and right context. In this way, we learn a bidirectional representation model to make recommendations by allowing each item in user historical behaviors to fuse information from both left and right sides. Extensive experiments on four benchmark datasets show that our model outperforms various state-of-the-art sequential models consistently.

users’ current interests are intrinsically dynamic and evolving, influenced by their historical behaviors. For example, one may purchase accessories (*e.g.*, Joy-Con controllers) soon after buying a Nintendo Switch, though she/he will not buy console accessories under normal circumstances.

To model such sequential dynamics in user behaviors, various methods have been proposed to make *sequential recommendations* based on users’ historical interactions [15, 22, 40]. They aim to predict the successive item(s) that a user is likely to interact with given her/his past interactions. Recently, a surge of works employ sequential neural networks, *e.g.*, Recurrent Neural Network (RNN), for sequential recommendation and obtain promising results [7, 14, 15, 56, 58]. The basic paradigm of previous work is to encode a user’s historical interactions into a vector (*i.e.*, representation of user’s preference) using a left-to-right sequential model and make recommendations based on this hidden representation.

Despite their prevalence and effectiveness, we argue that such left-to-right unidirectional models are not sufficient to learn optimal representations for user behavior sequences. The major limitation, as illustrated in Figure 1c and 1d, is that such unidirectional models restrict the power of hidden representation for items in the historical sequences, where each item can only encode the information from previous items. Another limitation is that previous unidirec-

BERT4Rec

$[i_1] \rightarrow [i_2] \rightarrow [i_3] \rightarrow [i_4]$

$[i_1] \rightarrow [\text{MASK}] \rightarrow [i_3] \rightarrow [i_4]$

Модель:

- ✓ видит прошлое
- ✓ видит и будущее
- ✓ угадывает пропуск

BERT4Rec

- **Cloze-задача:** $P(i \text{ masked} \mid \text{весь контекст})$
- Использует **энкодер** Трансформера с **двунаправленным (bidirectional) вниманием**
- Использует задачу **Item Masking** (заимствованную из BERT)

Шаг 1. Подготовка обучающих примеров

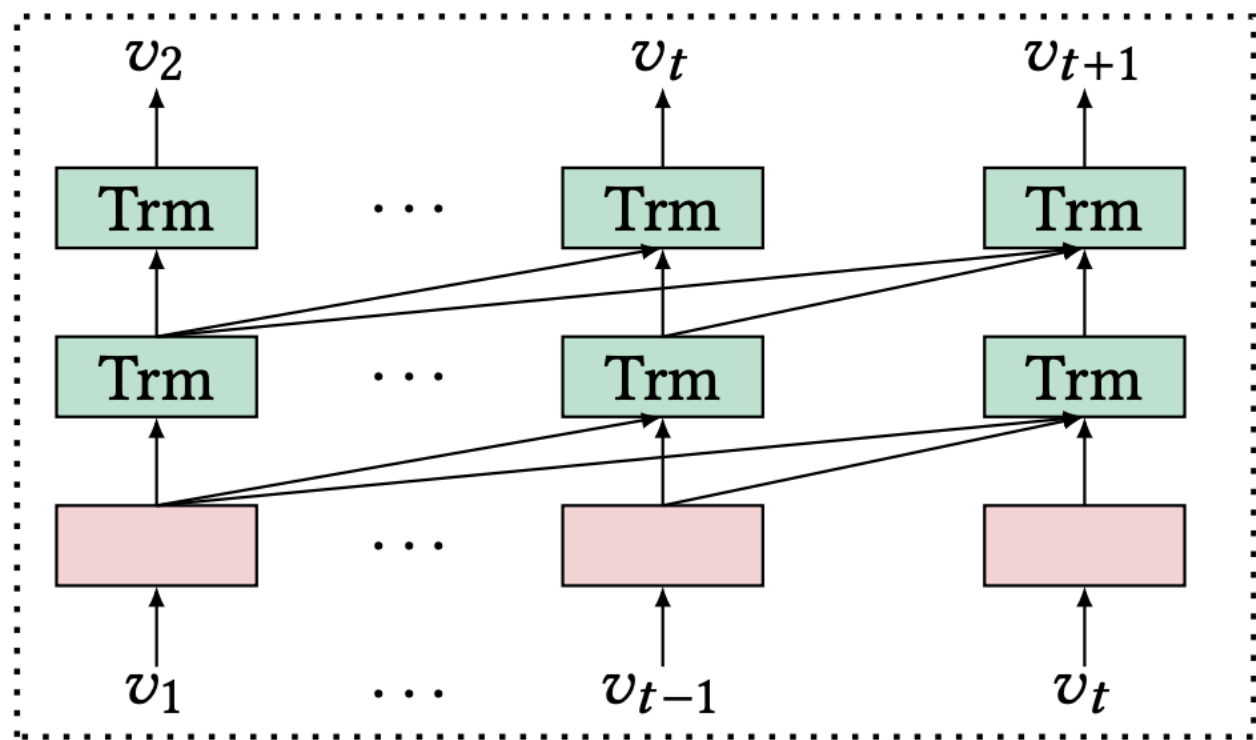
Шаг 2. Embedding слой

Шаг 3. Self-Attention (без causal mask, двунаправленный)

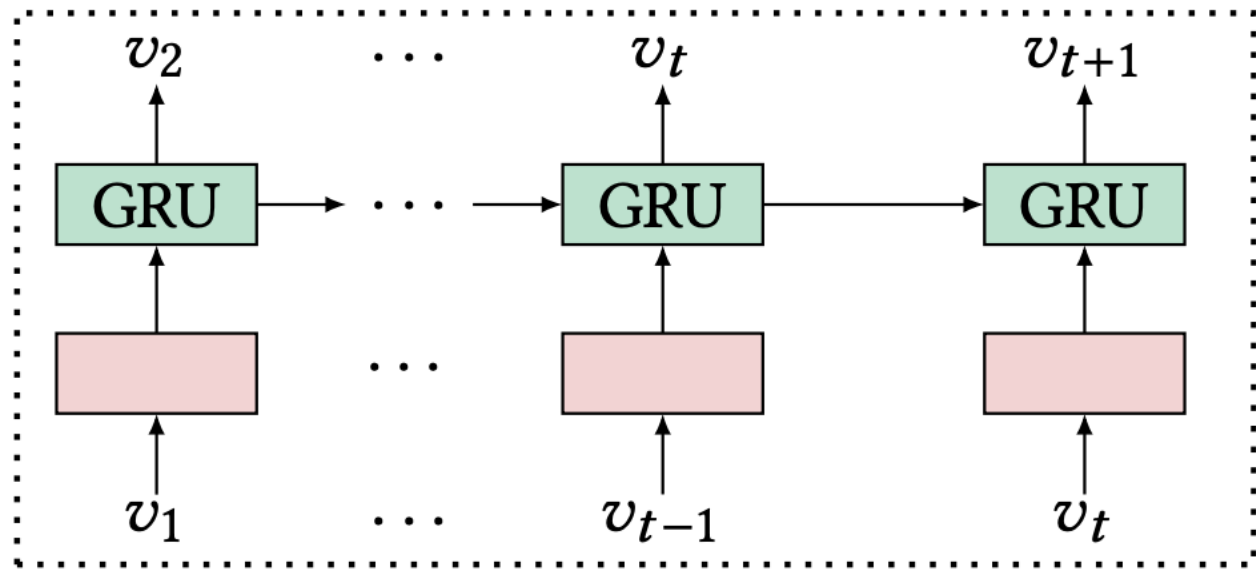
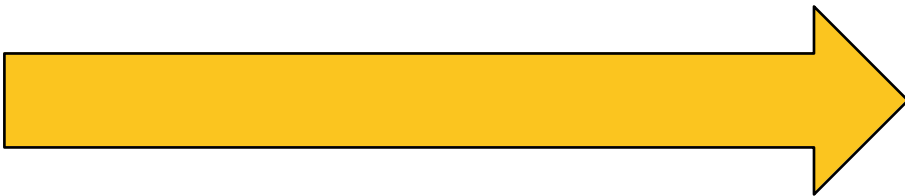
Шаг 4. Transformer блоки

Шаг 5. Предсказание MASK позиций

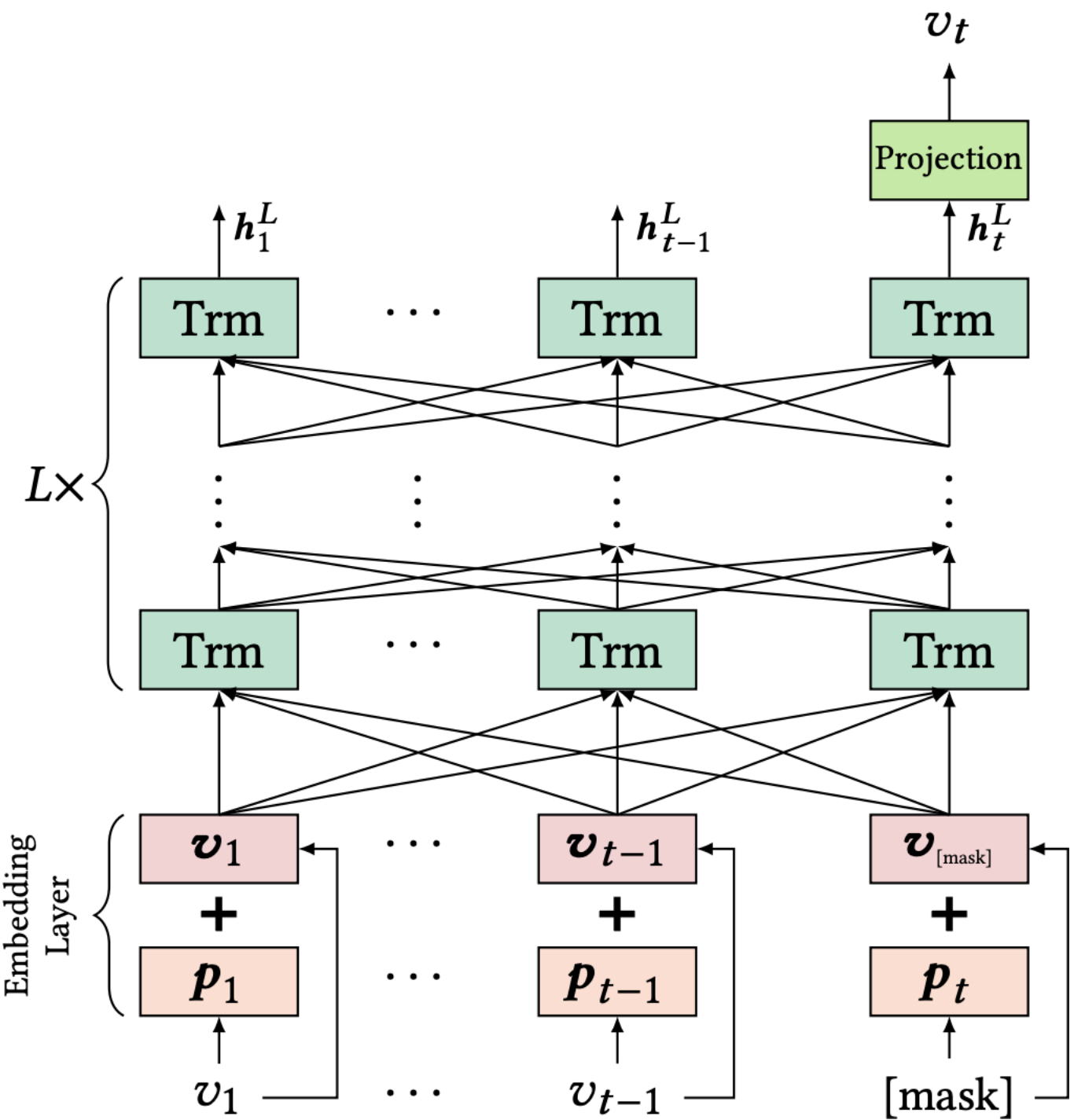
Introducing: BERT4Rec



(c) SASRec model architecture.



(d) RNN based sequential recommendation methods.



Introducing: BERT4Rec

- Можем смотреть на всю последовательность
- Маскируем случайные элементы в ней
- Для предсказания добавляем маску в конец последовательности
- Функция потерь – softmax

Input: $[v_1, v_2, v_3, v_4, v_5]$ $\xrightarrow{\text{randomly mask}}$ $[v_1, [\text{mask}]_1, v_3, [\text{mask}]_2, v_5]$
Labels: $[\text{mask}]_1 = v_2, \quad [\text{mask}]_2 = v_4$

The final hidden vectors corresponding to “[mask]” are fed into an output softmax over the item set, as in conventional sequential recommendation. Eventually, we define the loss for each masked input S'_u as the negative log-likelihood of the masked targets:

$$\mathcal{L} = \frac{1}{|S_u^m|} \sum_{v_m \in S_u^m} -\log P(v_m = v_m^* | S'_u) \quad (8)$$

where S'_u is the masked version for user behavior history S_u , S_u^m is the random masked items in it, v_m^* is the true item for the masked item v_m , and the probability $P(\cdot)$ is defined in Equation 7.

BERT4Rec: Метрики

Datasets	Metric	POP	BPR-MF	NCF	FPMC	GRU4Rec	GRU4Rec ⁺	Caser	SASRec	BERT4Rec	Improv.
Beauty	HR@1	0.0077	0.0415	0.0407	0.0435	0.0402	0.0551	0.0475	<u>0.0906</u>	0.0953	5.19%
	HR@5	0.0392	0.1209	0.1305	0.1387	0.1315	0.1781	0.1625	<u>0.1934</u>	0.2207	14.12%
	HR@10	0.0762	0.1992	0.2142	0.2401	0.2343	0.2654	0.2590	<u>0.2653</u>	0.3025	14.02%
	NDCG@5	0.0230	0.0814	0.0855	0.0902	0.0812	0.1172	0.1050	<u>0.1436</u>	0.1599	11.35%
	NDCG@10	0.0349	0.1064	0.1124	0.1211	0.1074	0.1453	0.1360	<u>0.1633</u>	0.1862	14.02%
	MRR	0.0437	0.1006	0.1043	0.1056	0.1023	0.1299	0.1205	<u>0.1536</u>	0.1701	10.74%
Steam	HR@1	0.0159	0.0314	0.0246	0.0358	0.0574	0.0812	0.0495	<u>0.0885</u>	0.0957	8.14%
	HR@5	0.0805	0.1177	0.1203	0.1517	0.2171	0.2391	0.1766	<u>0.2559</u>	0.2710	5.90%
	HR@10	0.1389	0.1993	0.2169	0.2551	0.3313	0.3594	0.2870	<u>0.3783</u>	0.4013	6.08%
	NDCG@5	0.0477	0.0744	0.0717	0.0945	0.1370	0.1613	0.1131	<u>0.1727</u>	0.1842	6.66%
	NDCG@10	0.0665	0.1005	0.1026	0.1283	0.1802	0.2053	0.1484	<u>0.2147</u>	0.2261	5.31%
	MRR	0.0669	0.0942	0.0932	0.1139	0.1420	0.1757	0.1305	<u>0.1874</u>	0.1949	4.00%
ML-1m	HR@1	0.0141	0.0914	0.0397	0.1386	0.1583	0.2092	0.2194	<u>0.2351</u>	0.2863	21.78%
	HR@5	0.0715	0.2866	0.1932	0.4297	0.4673	0.5103	0.5353	<u>0.5434</u>	0.5876	8.13%
	HR@10	0.1358	0.4301	0.3477	0.5946	0.6207	0.6351	<u>0.6692</u>	0.6629	0.6970	4.15%
	NDCG@5	0.0416	0.1903	0.1146	0.2885	0.3196	0.3705	0.3832	<u>0.3980</u>	0.4454	11.91%
	NDCG@10	0.0621	0.2365	0.1640	0.3439	0.3627	0.4064	0.4268	<u>0.4368</u>	0.4818	10.32%
	MRR	0.0627	0.2009	0.1358	0.2891	0.3041	0.3462	0.3648	<u>0.3790</u>	0.4254	12.24%
ML-20m	HR@1	0.0221	0.0553	0.0231	0.1079	0.1459	0.2021	0.1232	<u>0.2544</u>	0.3440	35.22%
	HR@5	0.0805	0.2128	0.1358	0.3601	0.4657	0.5118	0.3804	<u>0.5727</u>	0.6323	10.41%
	HR@10	0.1378	0.3538	0.2922	0.5201	0.5844	0.6524	0.5427	<u>0.7136</u>	0.7473	4.72%
	NDCG@5	0.0511	0.1332	0.0771	0.2239	0.3090	0.3630	0.2538	<u>0.4208</u>	0.4967	18.04%
	NDCG@10	0.0695	0.1786	0.1271	0.2895	0.3637	0.4087	0.3062	<u>0.4665</u>	0.5340	14.47%
	MRR	0.0709	0.1503	0.1072	0.2273	0.2967	0.3476	0.2529	<u>0.4026</u>	0.4785	18.85%

BERT4Rec: влияние маскирования

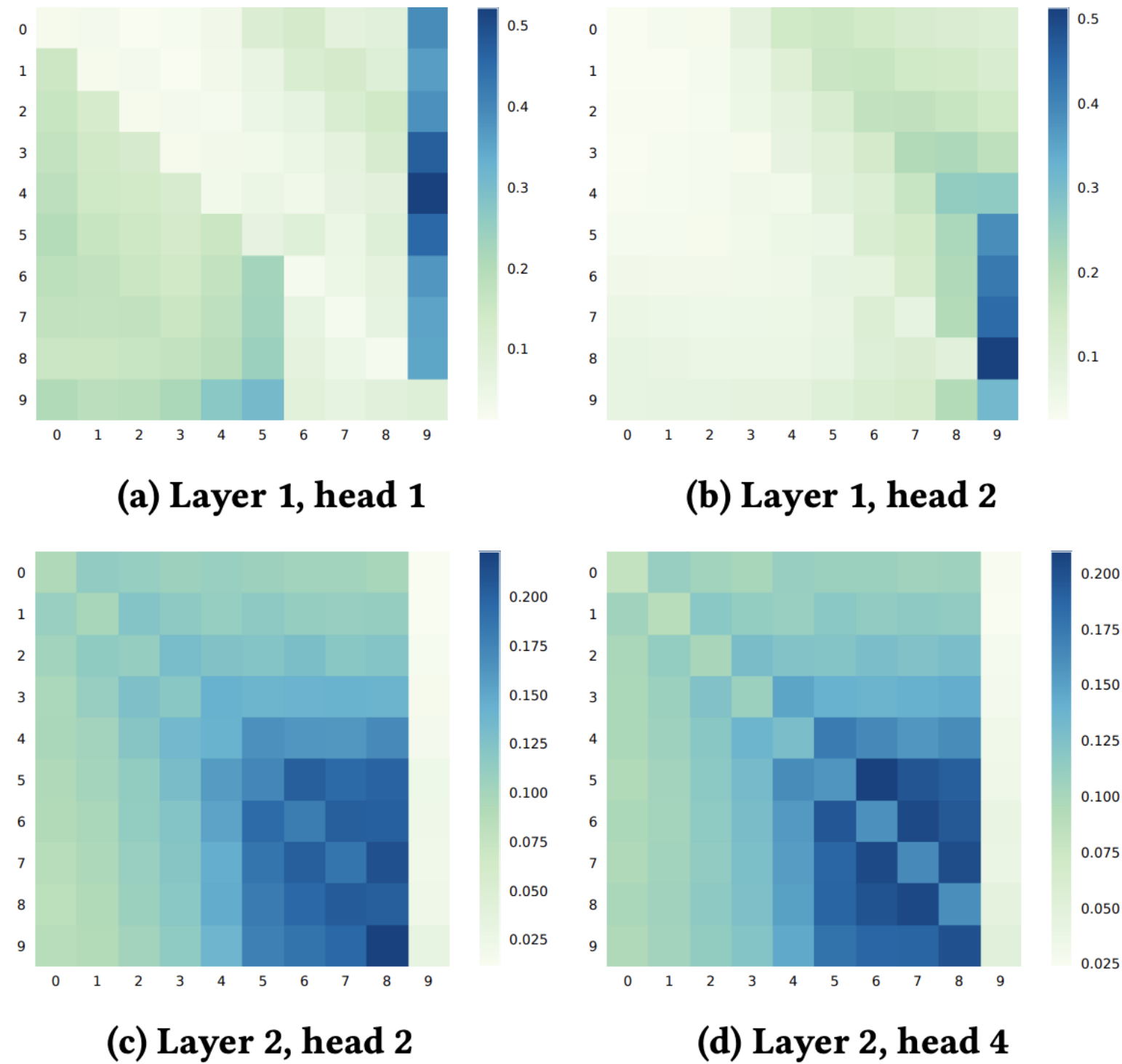


Figure 2: Heat-maps of average attention weights on Beauty, the last position “9” denotes “[mask]” (best viewed in color).

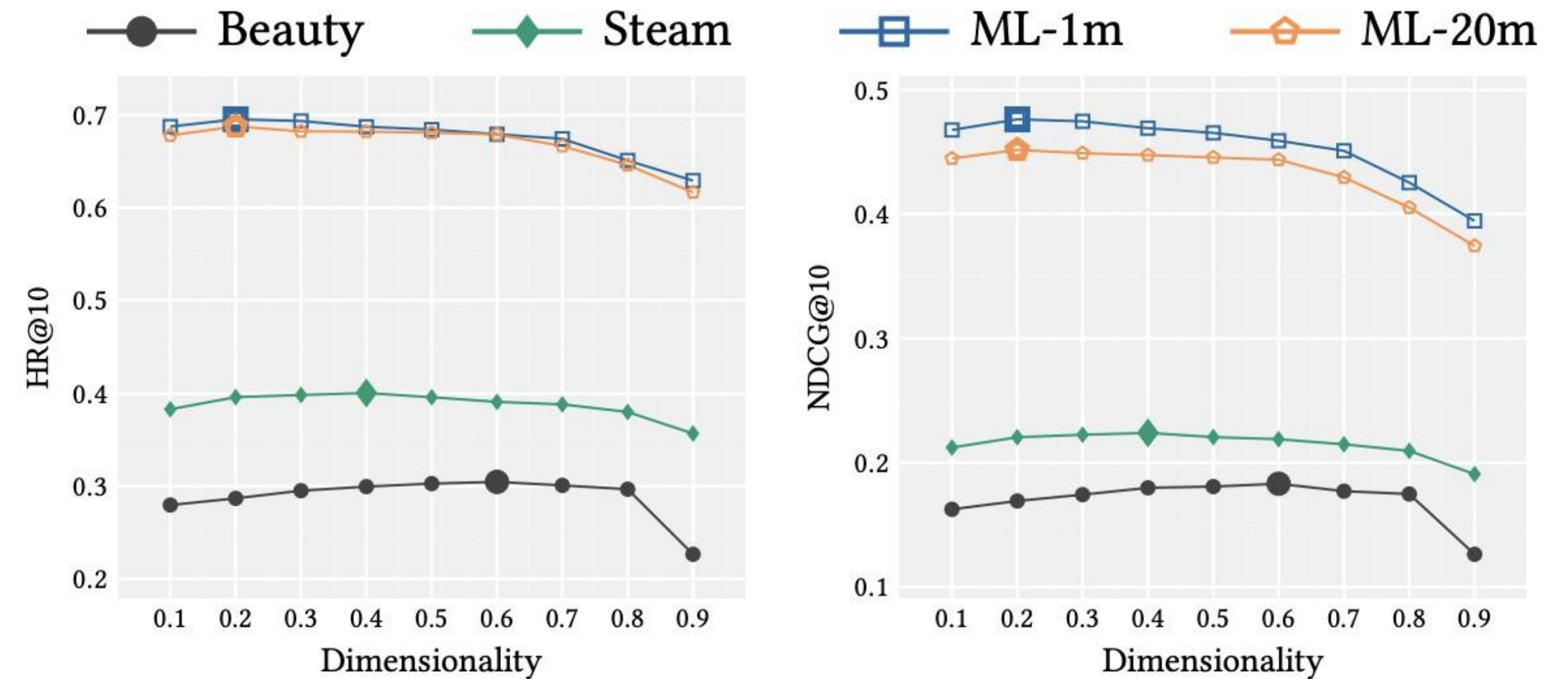


Figure 4: Performance with different mask proportion ρ on $d = 64$. Bold symbols denote the best scores in each line.

BERT4Rec: Детали

Table 5: Ablation analysis (NDCG@10) on four datasets. Bold score indicates performance better than the default version, while ↓ indicates performance drop more than 10%.

Architecture	Dataset			
	Beauty	Steam	ML-1m	ML-20m
$L = 2, h = 2$	0.1832	0.2241	0.4759	0.4513
w/o PE	0.1741	0.2060	0.2155↓	0.2867↓
w/o PFFN	0.1803	0.2137	0.4544	0.4296
w/o LN	0.1642↓	0.2058	0.4334	0.4186
w/o RC	0.1619↓	0.2193	0.4643	0.4483
w/o Dropout	0.1658	0.2185	0.4553	0.4471
1 layer ($L = 1$)	0.1782	0.2122	0.4412	0.4238
3 layers ($L = 3$)	0.1859	0.2262	0.4864	0.4661
4 layers ($L = 4$)	0.1834	0.2279	0.4898	0.4732
1 head ($h = 1$)	0.1853	0.2187	0.4568	0.4402
4 heads ($h = 4$)	0.1830	0.2245	0.4770	0.4520
8 heads ($h = 8$)	0.1823	0.2248	0.4743	0.4550

BERT4Rec: Не все так сладко

- Тяжело масштабировать. Почему?

BERT4Rec: Не все так сладко

- Тяжело масштабировать. Почему? Из-за **Softmax**

Какие мы выделили проблемы?

1. Сомнительный протокол тестирования
2. Плохая масштабируемость
3. Завышенные скоры у популярных айтемов

Завышенные скоры у топовых айтемов ведут к тому, что модель не учится отличать эти айтемы друг от друга (все их скоры приблизительно равны) и фокусируется на умении отличить лучшие от худших.

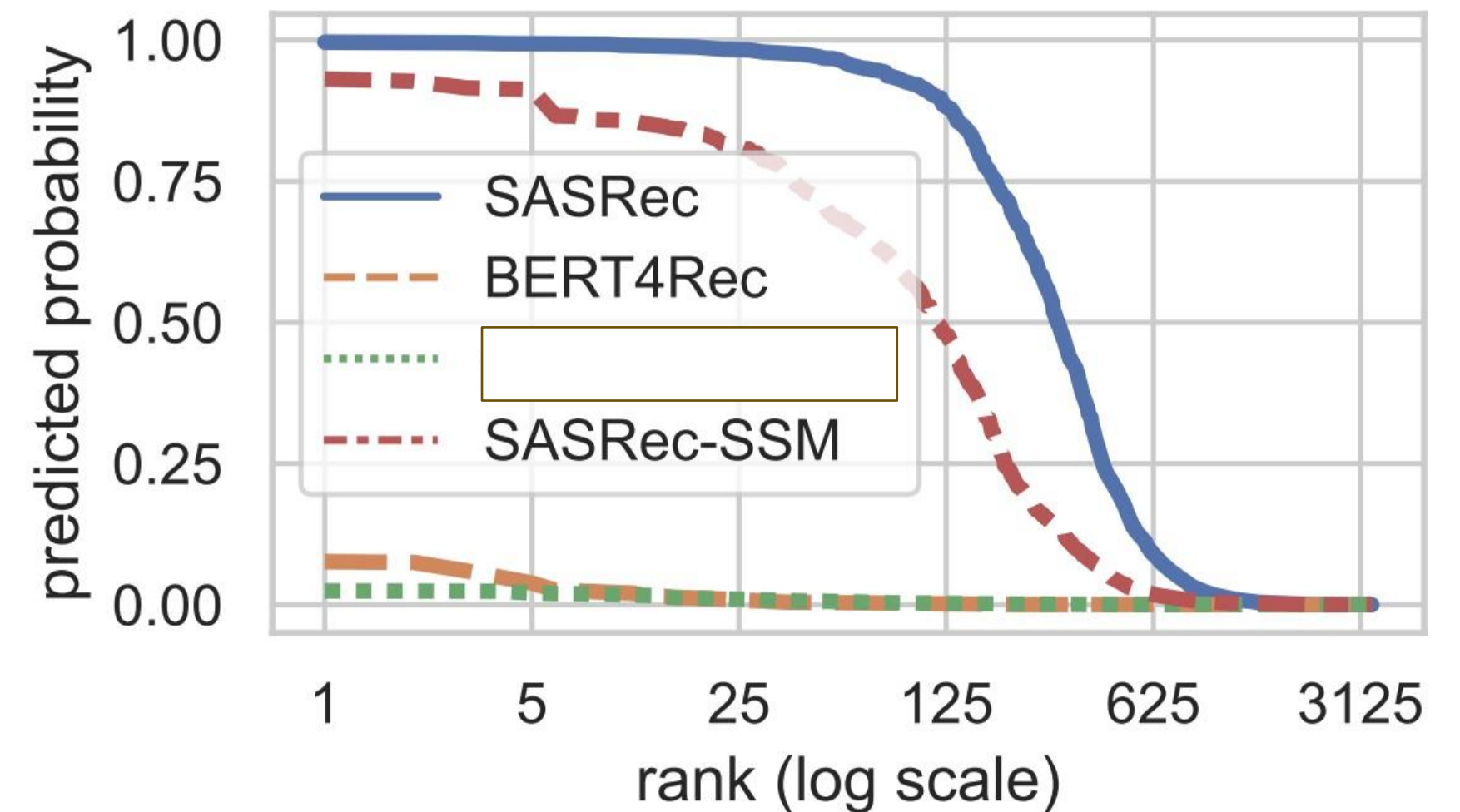


Figure 1: Predicted probability at different ranks for user 963 in MovieLens-1M. SASRec-SSM is a SASRec model trained with Sampled Softmax loss with 16 negatives.

SASRec: Обобщение

gSASRec: Reducing Overconfidence in Sequential Recommendation Trained with Negative Sampling

Aleksandr Petrov
University of Glasgow
United Kingdom
a.petrov.1@research.gla.ac.uk

Craig Macdonald
University of Glasgow
United Kingdom
craig.macdonald@glasgow.ac.uk

$$\mathcal{L}_{\text{gBCE}}^{\beta} = -\frac{1}{|I_k^{-}| + 1} \left(\log(\sigma^{\beta}(s_{i^{+}})) + \sum_{i \in I_k^{-}} \log(1 - \sigma(s_i)) \right) \quad \alpha = \frac{1}{|I| - 1}$$
$$\beta = \alpha \left(t \left(1 - \frac{1}{\alpha} \right) + \frac{1}{\alpha} \right)$$

SASRec: Обобщение

Dataset	Negative sampling and loss function→ Architecture↓	1 negative per positive; BCE Loss (as SASRec)	No negative sampling; Softmax Loss (as BERT4Rec)	Negative sampling and loss effect
ML-1M	SASRec	0.131	0.169	+29.0%*
	BERT4Rec	0.123	0.161	+30.8%*
	Architecture effect	-6.1%	-4.7%	
Steam	SASRec	0.0581	0.0721	+24.1%*
	BERT4Rec	0.0513	0.0746	+45.4%*
	Architecture effect	-11.7%*	+3.4%*	

BERT4Rec: дополнительные инженерные советы

- Masking strategy = критична
- Train / inference mismatch

Отличия

Характеристика	SASRec	BERT4Rec
Тип внимания	Однонаправленное	Двунаправленное
Маска	Causal mask	Нет
Обучение	Next-item prediction	Masked item prediction
Контекст	Только прошлое	Вся последовательность
Интерпретация	Генеративная модель	Восстановление пропусков
Поведение	Похожа на GPT	Похожа на BERT

Ограничения

1. Долгое обучение
2. Огромные каталоги товаров
3. Необходимость negative sampling
4. Большие embedding-тензоры
5. Медленный inference
6. Beyond-accuracy цели (diversity, novelty)

Проблемы и решения

Проблема (Limitation)	Суть проблемы	Предложенное решение (Contributions)
L2.1: Медленное обучение → часто underfitting	Из-за уникального набора товаров для каждого датасета нельзя использовать предобучение. Каждый раз модель учится с нуля, а BERT4Rec учится очень долго (до 16 часов на MovieLens-1M).	RSS (Recency Sampling of Sequences) – Новая стратегия обучения, которая вероятностно выбирает цели для предсказания, отдавая предпочтение недавним взаимодействиям. Это позволяет SASRec, обученному всего 1 час, достичь качества BERT4Rec, обученного 16 часов.
L2.2: Дорогой подсчет всех товаров при обучении → overconfident модель	В реальных системах миллиарды товаров. Прямой подсчет Softmax по всем товарам невозможен. Negative sampling приводит к overconfidence (модель переоценивает вероятность).	gBCE loss и gSASRec — Глава 5. Новая функция потерь (Generalized Binary Cross-Entropy), корректирующая влияние negative sampling через откалиброванные вероятности. Повышает калибровку и качество модели (+47% NDCG на Gowalla).

Проблемы и решения

Проблема (Limitation)	Суть проблемы	Предложенное решение (Contributions)
L2.3: Огромная таблица эмбедингов товаров	Таблица эмбедингов — самая "тяжелая" часть модели. При каталоге в миллиард товаров и размерности 128 — это 100+ млрд параметров, не влезающих в память.	RecJPQ Метод сжатия эмбедингов на основе Joint Product Quantization. Разбивает товары на "суб-ID", эмбединги хранятся только для них. Сжатие в 50 раз без потери качества.
L2.4: Дорогой подсчет всех товаров при инференсе	Даже если модель обучена, на этапе выдачи рекомендаций нужно снова посчитать скоринг для всех товаров. При каталоге в миллионы/миллиарды товаров это делает инференс неприменимо медленным для online-систем.	PQTopK и RecJPQPrune — Используя структуру RecJPQ (суб-ID), авторы предлагают алгоритмы для ускорения инференса. PQTopK предвычисляет скоринг суб-ID. RecJPQPrune использует динамический прунинг, достигая ускорения до 64x на Tmall (2.2M товаров).

Спасибо!